



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



DEDICATION

This work is dedicated to the **KEUMOE** family for their
countless encouragement and support throughout the
academic year.

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



ACKNOWLEDGMENTS

The drafting of this report would have not been possible without the numerous contributions of several individuals who dedicated their time and efforts to ensure its completion. Our heartfelt gratitude goes to all those people for their help, encouragements, advices, teachings, and precious time without which we would have not completed this project.

We would like to give thanks to the following people:

- The Resident Representative of AICS Cameroon, **Mr. Armand Claude ABANDA**, for his support, encouraging words, and valuable advice on how to approach innovatively any situation.
- The CEO of Realize, **Mr. NDELOGAKEH Daniel**, for offering us an opportunity to have a professional experience in a Tech company like Realize.
- Our Professional Supervisor, **Mr. TSEMBOM Percy Linkwe**, for his availability, advices and moral support throughout the internship period which lead to the realization of this project.
- Our Academic Supervisor, **Mr. NDIFOR Roger**, for his advices, tolerance, and moral support throughout the internship period.
- Our academic teachers, **Mr. AGBOR Anderson**, **Mr. MESSIO Ulrich** and **Mrs. TCHOUTOUO Isabelle** for their guidance and assistance in the realization of this document.
- To all my teachers at AICS Cameroon, for the knowledge they imparted, which has been essential through my academic journey.
- To the **REALIZE Team**, for their support and training during the internship period.
- My family most especially my **Mother** for their guidance, countless support and encouragement through the year.
- To all my classmates, for their collaborative spirit, opinions, and teamwork throughout the academic year.

We sincerely thank you all for your contributions.

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



CONTENT

DEDICATION	i
ACKNOWLEDGMENTS	ii
CONTENT	iii
LIST OF FIGURES	iv
LIST OF TABLES	vi
GLOSSARY	vii
ABSTRACT	viii
RESUME	ix
GENERAL INTRODUCTION	1
PART I: INSERTION PHASE	2
PART II: TECHNICAL PHASE	14
FILE 1: EXISTING SYSTEM	15
FILE 2: SPECIFICATION BOOK	24
FILE 3: ANALYSIS BOOK	41
FILE 4: CONCEPTION PHASE	73
FILE 5: REALIZATION PHASE	93
FILE 6: TEST OF FUNCTIONALITIES	106
FILE 7: INSTALLATION AND USER GUIDE	114
GENERAL CONCLUSION	122
ANNEXES	x
BIBLIOGRAPHY	xi
REFERENCES	xii
WEBOGRAPHY	xiii
TABLE OF CONTENT	xiv

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

LIST OF FIGURES

Figure 1 : Localization plan of Realize	7
Figure 2 : Functional organization of Realize	10
Figure 3: Project Gantt chart	34
Figure 4: UML 2.5 diagrams overview (source: https://d3n817fwly711g.cloudfront.net/uploads/2012/02/UML-Diagram-types-1-1024x658.png)	45
Figure 5: Two Track Unified Process (source: https://www.mysciencework.com/omniscience/pervasive-mobile-healthcare-system-basedon-cloud-computing)	48
Figure 6: Use case diagram formalism	51
Figure 7: General use case diagram	54
Figure 8: Activity diagram formalism	57
Figure 9: Authenticate activity diagram	60
Figure 10: Reserve Medication activity diagram.....	61
Figure 11: Search Medication activity diagram.....	62
Figure 12: Sequence diagram formalism	63
Figure 13	64
Figure 14: Authenticate sequence diagram.....	65
Figure 15: Reserve medication sequence diagram.....	66
Figure 16: Search medication sequence diagram.....	67
Figure 17: Communication diagram formalism.....	68
Figure 18: Communication diagram of Authenticate	70
Figure 19: Communication diagram of Reserve Medication.....	70
Figure 20: Communication diagram of Search Medication.....	71
Figure 21: Hardware diagram of the system.....	76
Figure 22: n-tier Architecture (source: https://www.pinterest.es/pin/752241943987565648/) ..	78
Figure 23: The MVC Architecture (source: https://www.freecodecamp.org/news/the-model-view-controller-	79



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



Figure 24: Class diagram formalism.....	80
Figure 25: Object diagram formalism.....	83
Figure 26: State Machine diagram formalism	86
Figure 27: Package diagram formalism	89
Figure 28: Swift Meds package diagram	91
Figure 29: Deployment diagram formalism.....	99
Figure 30: Swift Meds deployment diagram	101
Figure 31: Component diagram formalism.....	102
Figure 32: Swift Meds component diagram.....	104
Figure 33: User registration code test snippet.....	110
Figure 34: Authentication test code snippet.....	111
Figure 35: Registration and Authentication unit test	112
Figure 36: frontend configuration of the .env file.....	118
Figure 37: backend configuration of the .env file.....	118

LIST OF TABLES

<i>Table 1 : Hardware Resources</i>	11
<i>Table 2 : Software Resources</i>	11
<i>Table 3: Limitations, consequences and proposed solutions</i>	20
<i>Table 4: Chronogram of Activities</i>	33
<i>Table 5: Software Resources</i>	35
<i>Table 6: Hardware Resources (source: mercurial.cm)</i>	36
<i>Table 7: Human Resources (source: Mercurial 2025)</i>	36
<i>Table 8: Global Estimation</i>	37
<i>Table 9: List of Participants</i>	39
<i>Table 10: Comparison between MERISE and UML</i>	46
<i>Table 11: Component of a Use case diagram</i>	52
<i>Table 12: Primary actors of our system</i>	53
<i>Table 13: Secondary actors of our system</i>	53
<i>Table 14: Textual description formalism</i>	55
<i>Table 15: Component of an Activity diagram</i>	58
<i>Table 16: Component of sequence diagram</i>	64
<i>Table 17: Component of a Communication diagram</i>	69
<i>Table 18: Component of a Class diagram</i>	81
<i>Table 19: Component of an Object diagram</i>	84
<i>Table 20: Component of a State machine diagram</i>	87
<i>Table 21: Component of a Package diagram</i>	90
<i>Table 22: Used Technology</i>	96
<i>Table 23: Component of a Deployment diagram</i>	100
<i>Table 24: Component of a Component diagram</i>	103



GLOSSARY

- ❖ AICS: African Institute of Computer Science
- ❖ HTML: HyperText Markup Language
- ❖ CSS: Cascading Style Sheets
- ❖ IT: Information Technology
- ❖ HTTPS: HyperText Transfer Protocol Secure
- ❖ RUP: Rational Unified Process
- ❖ XP: Extreme Programming
- ❖ ERDs: Entity Relationship Diagrams
- ❖ DFDs: Data Flow Diagrams
- ❖ SSL: Secure Socket Layer
- ❖ API: Application Programming Interface
- ❖ UAT: User Acceptance Testing
- ❖ KPIs: Key Performance Indicators
- ❖ 2TUP: Two Track Unified Process
- ❖ UML: Unify Modeling Language
- ❖ MVC: Model View Controller
- ❖ DBMS: Database Management System



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



ABSTRACT

The project theme “**DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING (Swift Meds)**” addresses the critical challenge of locating and accessing medications efficiently by developing a web-based pharmacy system for real-time medication availability and fast dispensing. In many regions, patients struggle to find required medications during emergencies or daily needs due to scattered pharmacy locations and lack of centralized real time information. To solve this problem, we designed and implemented an intermediary platform connecting clients with pharmacy agencies, enabling users to search, locate, and reserve medications in real time. The system leverages a real-time database to provide up-to-date information on drug availability across multiple pharmacies, allowing users to identify the nearest location where their medication is in stock. Additionally, a premium reservation feature ensures that busy or emergency-bound patients can secure their medications for later pickup. By optimizing the medication search and dispensing process, **Swift Meds** enhances efficiency, reduces time wastage, and improves healthcare accessibility. The platform highlights its user-friendly interface, secure database management, and impact on streamlining pharmacy operations. This demonstrates that SwiftMeds significantly improves medication accessibility, offering a scalable solution for modern healthcare challenges.

KEYWORDS:

- Real-time availability
- Reservation system
- Accessibility
- Location

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



RESUME

Le projet «**DÉVELOPPEMENT D'UN SYSTÈME PHARMACEUTIQUE POUR LA DISPONIBILITÉ DES MÉDICAMENTS EN TEMPS RÉEL ET UNE DISPENSATION RAPIDE (Swift Meds)**» vise à résoudre le défi critique de localisation et d'accès aux médicaments en développant une plateforme web permettant un suivi en temps réel et une dispensation rapide. Dans de nombreuses régions, les patients éprouvent des difficultés à trouver les médicaments nécessaires, surtout en cas d'urgence, en raison de la dispersion des pharmacies et du manque d'informations centralisées en temps réel. Pour remédier à ce problème, nous avons conçu et implémenté une plateforme intermédiaire reliant les clients aux pharmacies, permettant aux utilisateurs de rechercher, localiser et réserver des médicaments en temps réel. Le système s'appuie sur une base de données temps réel pour fournir des informations actualisées sur la disponibilité des médicaments dans différentes pharmacies, aidant ainsi les utilisateurs à identifier le point de vente le plus proche disposant du traitement recherché. De plus, une fonctionnalité premium de réservation permet aux patients pressés ou confrontés à une urgence de réserver leurs médicaments pour un retrait ultérieur. En optimisant le processus de recherche et de dispensation, **Swift Meds** améliore l'efficacité, réduit les pertes de temps et facilite l'accès aux soins de santé. Cette plateforme se distingue par son interface conviviale, sa gestion sécurisée des données et son impact sur la rationalisation des opérations pharmaceutiques. Les résultats démontrent que SwiftMeds améliore significativement l'accessibilité aux médicaments, offrant ainsi une solution évolutive aux défis actuels de la santé.

MOTS-CLES:

- Disponibilité en temps réel
- Système de réservation
- Accessibilité
- Localization



GENERAL INTRODUCTION

In today's fast-paced world, access to essential medications remains a critical challenge for many individuals, particularly during emergencies or in regions with limited pharmacy coverage. Patients often face difficulties locating specific drugs due to the lack of a centralized, real-time system that tracks medication availability across multiple pharmacies. This inefficiency leads to wasted time, increased stress, and, in some cases, delayed medical treatment which potentially result in worse health outcomes. As part of my academic journey at AICS Cameroon, I undertook a three month internship session at REALIZE Tech a company which offers software development services with IT features. During this session, I was assigned the project theme “**DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING**” (**Swift Meds**), this project by leveraging modern technologies; it ensures that users can quickly find the nearest pharmacy stocking their required medication. This report is divided into several sections:

1. **The Insertion Phase:** It involves the introduction of the company.
2. **The Existing System:** Here, we present the current system in place.
3. **The Specification Book:** Outlines the needs of the system and identify the various project requirements.
4. **The Analysis Phase:** A presentation of the analysis methods and diagrams utilized for the system realization.
5. **The Conception Phase:** A detailed description of the project's design, highlighting its core components.
6. **The Realization Phase:** An overview of the implementation process for the proposed solution.
7. **Test of Functionalities:** Here, we will explore the applications' functionalities.
8. **The User Guide:** Instructions on how to install and use the system for users.

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



PART I: INSERTION PHASE

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



PREAMBLE

This section of the report will cover details of how we were welcomed in the Host Company, presentation, organization and brief introduction to our project.

CONTENT

INTRODUCTION

- I. WELCOME AND INSERTION
- II. GENERAL PRESENTATION OF THE COMPANY
- III. ORGANIZATION OF THE COMPANY
- IV. HARDWARE AND SOFTWARE RESOURCES
- V. BRIEF PRESENTATION OF PROJECT THEME

CONCLUSION

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



INTRODUCTION

The insertion phase within a company serves as a period for us to familiarize ourselves with the working environment, meet the staff, and connect with fellow interns. In this section, we will start by reflecting on our initial two weeks at the company, discussing the warm welcome we received and how we began to adapt to our internship setting. Next, we will delve into the company's history, uncover its mission, explore its main activities, and highlight some of its significant achievements. We will also gain an understanding of the company's administrative and functional structure, which enables it to operate efficiently and achieve its objectives. Additionally, we will examine the hardware equipment utilized by the organization, as well as the software tools that support its daily operations. Finally, we will introduce the theme we selected for our internship period and provide a brief overview of it.



I. WELCOME AND INSERTION

The welcoming and integration event took place on the 01th of July at 8:00 am. Upon arrival, we were warmly welcomed by Mr. Mengot and Mrs. Tatang, senior developer and analyst researcher respectively, who delivered an inspiring welcoming speech. Having my first interned at REALIZE, it was a new experience, though it was refreshing to connect with a professional team. We proceeded with a presentation and a meeting focused on discussing the challenges we faced during the coding test, which was a prerequisite for our internship at the company. The meeting also included an overview of the enterprise's structure, its internal policies, and the specific workstations assigned to each intern. Though it was new experience, I was able to rapidly adapt myself within the company's environment. Mr. Tanue concluded by encouraging us to strive for excellence and offered his support for any issues we might encounter.

After settling into our designated workstations, we were gently introduced into the way to visualize a problem in the society, how to understand the core points of the problem, and how to apply our IT knowledge to propose a solution this problem. The next day, we were promptly assigned our first task for the day. During the first week, we conducted a verification of concepts using fundamental HTML and CSS to ensure everyone was on an even playing field before progressing. We then engaged in discussions about project ideas, where we were encouraged to propose our own. I did my best to contribute and refine these ideas. Subsequently, we were tasked with researching these ideas, as the enterprise valued our initiative and did not impose any themes on us.

In the second week, we focused on JavaScript concept verification exercises to assess our understanding and reasoning skills. We held a detailed discussion with our professional supervisor about the specification book and its various components, where he outlined what was expected of us in each segment. We also continued developing our project ideas into a concrete plans.

II. GENERAL PRESENTATION OF THE COMPANY

A. Background

Realize is a Cameroon based non-governmental tech start-up institute founded by **Mr. NDELOGAKEH Daniel** in 2022 which proposes IT solutions and empowers the use of new technologies in Cameroon.

B. Missions, Visions and Activities of Realize

1. Missions:

The mission of Realize mainly relates to the empowerment and the perpetuation of Technology use. These missions include:

- Design and realize mobile-oriented software for companies and individuals.
- Assist newborn companies for quick growth using computer sciences.
- Provide training and certifications to improve qualified human resources in many fields of study.
- Take an active part in the sustainable development of the world through innovative solutions and virtual reality.

2. Visions:

At Realize, we believe that the true potential of every organization and individual can be unlocked through the harmonious fusion of analysis, development, realization, and data analytics. Our vision propels us forward, guiding us to make a lasting impact on the digital landscape and shape a future where innovation knows no bounds.

3. Activities:

The activities of Realize range from computer sciences, engineering, and training. We can outline the following:

- Conception realization, and hosting of websites;

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



- Software development and maintenance;
- Training in Software related fields;
- Conception and realization of multimedia;
- IT consulting and innovation
- IT support

4. Localization:



Figure 1 : Localization plan of Realize



III. ORGANIZATION OF THE COMPANY

A. Administrative Organization

Realize if administratively organized as follows:

1. The General Management

This is the highest level of the company, which takes care of the following:

- Proper functioning of each department;
- Define project strategies;
- Provide leadership and guidance to the company's employees.
- Makes critical decision that could affect the company's operations or reputation.
- Severe as a point of contact for key stakeholders such as investors etc.

2. Human Resource Department

This department is in charge of the following:

- Recruitment and Hiring
- Acts as a liaison between employees and management;
- Manages employee compensation and benefits programs;
- Develops and implements company policies and procedures, ensuring compliance with employment laws and regulations.
- Manages administrative tasks related to employee records, data management, and HR systems

3. Communication Department

This department is in charge of the following:

- Handles public relations activities, which involve managing the company's reputation and image;



- Responsible for crafting and delivering messages to external stakeholders such as customers etc.;
- It fosters effective communication within the company;
- They create and manages content across different platforms and channels;

4. Department of Financial Affairs

This department is in charge of the following:

- Responsible for developing and managing the company's financial plans and budgets;
- Prepare and presents accurate and timely financial reports to management, stakeholders, and regulatory authorities;
- Responsible for managing cost and expenses within the organization;
- Ensures compliance with tax laws and regulations. Etc.

5. Technical Department

This department is in charge of the following:

- Responsible for managing the company's technological infrastructure, including networks, hardware;
- Play a role in managing and maintaining the company's data;
- Responsible for implementing and maintaining cyber security measures to protect the company's digital assets from potential threats;
- Interacts with technology vendors and manages relationships with external service providers. Etc.

6. Software Engineering Department

This department is in charge of the following:

- Primarily responsible for developing software applications and systems;
- Responsible for maintaining and supporting software application throughout their lifecycle;
- Evaluation and realization of projects etc.



B. Functional Organization of REALIZE

The functional branch of Realize is organized as follows

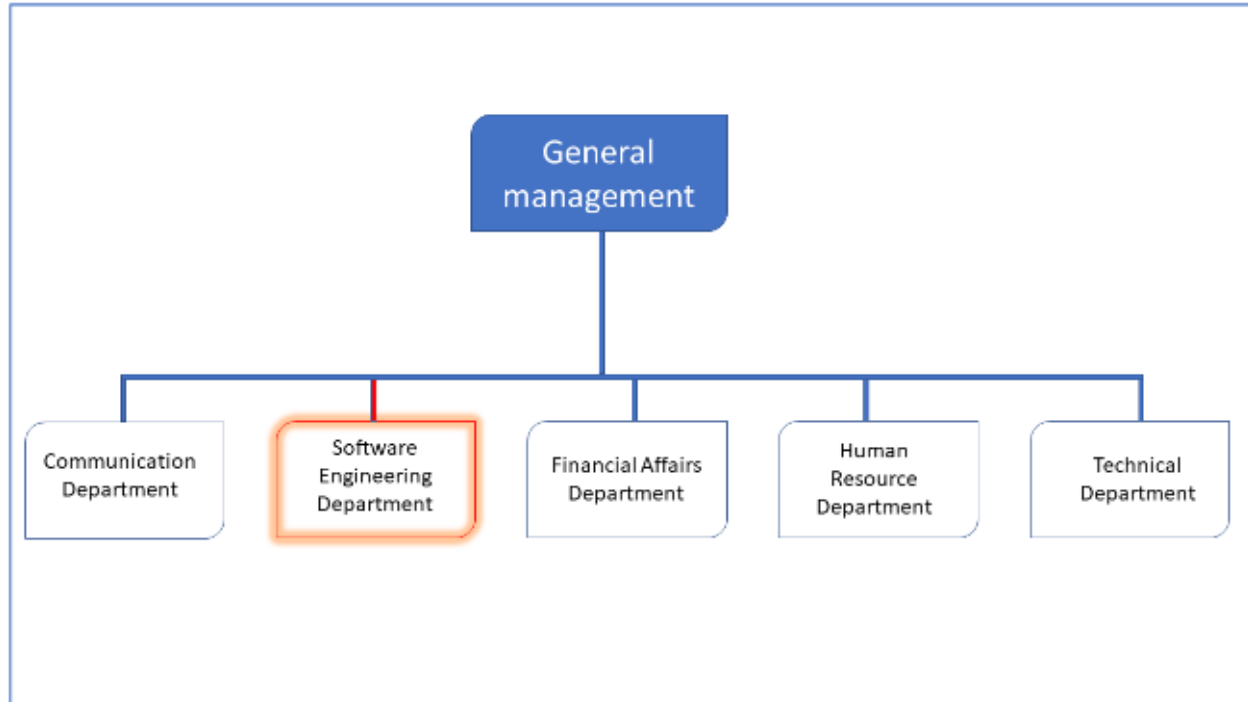


Figure 2 : Functional organization of Realize

IV. HARDWARE AND SOFTWARE RESOURCES

A. HARDWARE RESOURCES

Table 1 : Hardware Resources

No	Designation	Quantity	Characteristics	Observation
1	MacBook Pro	1	APPLE	Good
2	Modem	1	CAMTEL	Good
3	LCD Screen	1	HP	Good
4	Training Equipment	/	/	
5	Office Furniture	/	/	

B. SOFTWARE RESOURCES

Table 2 : Software Resources

Designation	Software
Operating System	Windows 11, Parrot Linux, Mac OS
Integrated Development Environment(IDE)	Visual Studio Code
Database Management System(DBMS)	MongoDB, PostgreSQL, MySQL
Web Browser	Google Chrome, Microsoft Edge
Document Editor	Microsoft Office Word



V. BRIEF PRESENTATION OF PROJECT THEME

After arriving at REALIZE, at the end of the initial two weeks, we already had an idea on a problem that affect the society, we identified an opportunity to create a project that could make a significant impact. We proposed the theme “**DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING**”. This system, which we have named **Swift Meds**, aims to be a web-based pharmacy system designed to provide real-time medication tracking and fast dispensing. This platform serves as an intermediary between patients and pharmacies, offering an efficient solution for locating and reserving medications with ease.



CONCLUSION

Our integration at REALIZE that lasted for two weeks, that was a new experience to our catalog, during which we experienced an exceptional environment that encourages hard work, competence and qualities that many aspiring professionals seek. We had the chance to meet and connect with the staff and fellow interns at REALIZE, promoting a sense of community and collaboration. This welcoming atmosphere promoted open communication and facilitated knowledge sharing. Finally, we choose the project theme “**DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING**”. This theme represent our will of resolving the problem people faced when the urgently need a medicine and goes along with the motives of REALIZE, the proposition of an innovative solution which implement IT to it.



PART II: TECHNICAL PHASE



FILE 1: EXISTING SYSTEM



PREAMBLE

The existing system is a document that provides a view of the system currently implemented, that is outlined how the different activities and operation are carry out, also provides a deep understanding of this system associated to the various limitations, the problems that result from these and the solution we propose.

CONTENT

INTRODUCTION

- I. PRESENTATION OF THE THEME
- II. STUDY OF THE EXISTING SYSTEM
- III. LIMITATIONS OF THE EXISTING SYSTEM
- IV. PROBLEMATICS
- V. PROPOSED SOLUTION

CONCLUSION



INTRODUCTION

As engineers, our role in society revolves around problem-solving aimed at making life easier for everyone. To develop effective solutions, we must first ask ourselves critical questions to understand why the problem exists and how it is currently being addressed. The existing system refers to the framework currently in place for managing the tasks related to our project theme. Gaining a thorough understanding of this system is crucial for identifying the problems that need to be addressed. It is essential to consider this system carefully before proposing a solution that will enhance or replace the current approach.



I. PRESENTATION OF THE THEME

Our project theme is “**DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING**”. This system, named **Swift Meds**, aims to provide system which acts an intermediary between patients in needs of medications and the pharmacies which are medication supply points. It aims to be a web-based pharmacy system designed to provide real-time medication tracking and fast dispensing.

Swift Meds is designed to shatter these barriers by creating a centralized, real-time ecosystem. The system will leverage real-time database technology to provide a live view of medication availability across a network of partnered pharmacies. Through an intuitive user interface, patients will be able to instantly search for a specific drug, view its price, see which pharmacies have it in stock, and identify the closest option using integrated geolocation services. Furthermore, to streamline the process and guarantee access, **Swift Meds** will incorporate a secure reservation and payment system. This allows users to reserve a medication for immediate or future pickup, ensuring it is held for them and ready upon arrival, thus transitioning the dispensing process from uncertain to guarantee.

For pharmacies, **Swift Meds** is not just a listing service but a powerful management tool. It provides a dedicated dashboard to update inventory, track reservations, manage patient relationships, and gain valuable insights into local demand trends, enabling better stock optimization and reducing losses from expired goods.

In summary, this project aims to transform the pharmaceutical access model in Yaoundé by deploying a technology driven solution that addresses the critical flaws of information delay and inaccessibility, thereby enhancing healthcare efficiency, reducing operational burdens on pharmacies, and, most importantly, providing residents with a reliable tool to meet their medical needs swiftly and effectively.



II. STUDY OF THE EXISTING SYSTEM

In the urban landscape of Yaoundé, Cameroon, access to essential medication remains a significant and daily challenge for a large portion of the population. The current process for locating and acquiring prescribed drugs is predominantly manual, relying on an inefficient cycle of phone calls and physical travel from one pharmacy to another. This "pharmacy run-around" is not only time consuming and economically draining due to accumulated transportation costs, but it also poses a serious risk during medical emergencies where time is a critical factor. The core of this problem lies in the profound information asymmetry between patients and pharmacy agencies; there exists no centralized, real-time system to bridge this gap, leaving vital medication sitting on shelves unknown to those who need it most.

For the Patient, the process is one of frustration and uncertainty. Needing a specific medication, a patient must embark on a tedious "pharmacy run-around," making endless phone calls or visiting multiple locations in person. Each stop is a gamble; even a positive phone confirmation offers no guarantee the medication will still be available upon arrival. This process consumes invaluable time, incurs significant transportation costs, and turns a simple medical need into a source of major stress, especially during emergencies.

For the Pharmacy, the current system is equally inefficient and disruptive. Pharmacy staff is constantly interrupted by phone calls asking the same questions about stock availability, pulling them away from their core duties of serving in-store customers and managing inventory. This manual, repetitive inquiry process is a significant drain on productivity.

Existing attempts at solutions, such as simple directory apps or static listings, have proven inadequate. They often fail because they depend on pharmacists to manually update stock levels a task that is easily forgotten in a busy environment leading to out dated and unreliable information that erodes user trust. Consequently, the market is ripe for a robust, intelligent, and automated system that can dynamically synchronize supply with demand.

III. LIMITATIONS OF THE EXISTING SYSTEM

Table 3: Limitations, consequences and proposed solutions

LIMITATIONS	CONSEQUENCES	PROPOSED SOLUTION
Inefficient Emergency Response	Heightened stress and significantly worsened health outcomes during critical medical situations.	Geolocation powered search to instantly find the nearest available medication, turning hours of search into seconds.
No Real-Time Inventory Visibility	The frustrating “pharmacy run-around” wasting time, money, and energy visiting or calling multiple locations.	A centralized real-time database that provides instant, accurate visibility of medication availability across all partnered pharmacies.
Poor Inventory Management	Frequent stock outs of high-demand medications. Financial losses from both overstocking and under stocking.	A dashboard for pharmacies that provides insights into popular medication and demand patterns, enabling smarter stocking decisions and reducing waste.
Manuel and Error-Prone Processes	Incorrect information received over the phone leads to wasted trips and loss of trust.	A digital, self-service platform that gives patients accurate information directly, freeing pharmacy staff
Lack of a Reservation System	No guarantee that a found medication will still be available upon arrival, making the entire search process uncertain.	A secure reservation system that allows patients to lock and guarantee their medication for pickup, and allows pharmacies to prepare orders ahead of time.



IV. PROBLEMATICS

Upon examining the current system, we discover some limitations. This brings us to a general question that we must ask ourselves concerning the system in place: **HOW CAN WE ENSURE THE AVAILABILITY AND RAPID ACCESSIBILITY OF MEDICATIONS DURING EMERGENCIES OR DAILY NEEDS?**

V. PROPOSED SOLUTION

Based on the existing challenges we encountered in the current system, we proposed the development of a comprehensive web platform, **Swift Meds**. The solution encompasses the following key points:

i. Centralized Web Platform:

Creation of a comprehensive, real-time pharmacy system that serves as a single source of truth for medication availability.

ii. Real-Time Medication Availability:

Implementation of a live inventory synchronization system, providing clients with instant, accurate visibility of medication stock across all partnered pharmacies.

iii. Facilitation of Fast Dispensing:

Streamlining the journey from search to pick up through integrated reservation and payment features, significantly reducing wait times.

iv. Critical Intermediary:

The system functions as an essential digital bridge, directly connecting patients in need with pharmacies that have the required supplies.

v. User-Friendly Intuitive Interface:

Delivery of an accessible and easy-to-navigate interface for both clients and pharmacists, ensuring the system is adoptable by all user groups.



CONCLUSION

In conclusion, Swift Meds directly tackles the critical challenges of medication access. By providing real-time visibility, enabling secure reservations, and connecting patients directly to pharmacy inventories, the project promises to save time, reduce stress, and ensure that vital healthcare is both accessible and reliable for everyone.

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



FILE 2: SPECIFICATION BOOK

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



PREAMBLE

The Specification book is a document which is been established by the customer needing a particular product and the producer (engineer) who is to create the product, which is to be respected to its fullness during the realization of the product. From this, we obtain the major importance of the specification book.

CONTENT

INTRODUCTION

- I. CONTEXT AND JUSTIFICATION
- II. OBJECTIVES OF THE PROJECT
- III. EXPRESSION OF NEEDS
- IV. PROJECT PLANNING
- V. ESTIMATED COST OF THE PROJECT
- VI. PROJECT CONSTRAINTS
- VII. LIST OF PARTICIPANTS AND DELIVERABLES

CONCLUSION



INTRODUCTION

The specification document for our report serves to clarify our theme, enhancing our understanding and increasing the chances of its success. To define the project's scope, we will outline the context of our theme. From this context, we will identify the specific problems we aim to address throughout the project. After presenting our proposed solution, we will discuss the objectives we have established for the project. We will also examine the needs our system will fulfill, considering both functional and non-functional aspects. Next, we will assess the estimated financial resources required for our project and outline a timeline to ensure we complete it on schedule. Finally, we will address the expectations for our project deliverables by the end of the project.



I. CONTEXT AND JUSTIFICATION

A. CONTEXT

In urban centers across Cameroon, such as Yaoundé, accessing essential medications remains a pervasive daily challenge that exposes critical weaknesses in local healthcare infrastructure. The existing medication procurement system is characterized by extreme fragmentation, where pharmacies operate as isolated silos with no coordinated communication or inventory sharing mechanisms. This structural deficiency forces patients and caregivers into a labour-intensive process of manually contacting or physically visiting multiple pharmacies in succession an inefficient practice often called the "pharmacy run-around." Each unsuccessful attempt consumes limited financial resources through transportation costs, drains time, and compounds the anxiety of seeking treatment. The problem becomes particularly severe during medical emergencies, where delays in locating critical drugs can directly lead to deteriorating health outcomes, avoidable complications, or even preventable fatalities. At the heart of this systemic breakdown lies fundamental information disconnects, patients cannot locate available medicine, and pharmacies cannot effectively signal their supply to those in need.

This access crisis is further exacerbated by the absence of any centralized or real-time platform capable of aggregating and distributing medication availability data. Patients are left to rely on out dated word-of-mouth recommendations, inaccurate directory listings, or sheer luck. Pharmacies, meanwhile, remain unaware of acute demand signals in their community, preventing them from responding proactively to urgent needs. The resulting inefficiencies do not merely represent an inconvenience they constitute a serious public health risk and a significant drain on household economies, particularly for those managing chronic conditions requiring regular medication. Without intervention, this cycle of waste and risk will continue to undermine healthcare delivery for the urban population.



B. JUSTIFICATION

The development of the Swift Meds system is not merely a technological upgrade but a necessary intervention justified by a critical convergence of needs and opportunities. The profound inefficiencies of the current model have created a system that is unsustainable from an operational, public health, and economic standpoint. Therefore, this project is propelled by four core imperatives that validate its urgency and strategic importance:

1. **Operational Necessity:** The transition from a manual, high-touch, error-prone model to an automated, algorithmic supply-demand matching system is not an innovation but a fundamental modernization required for basic healthcare efficiency.
2. **Public Health Imperative:** The project is a direct intervention to mitigate a measurable public health risk. The time between prescription and acquisition is a critical variable in treatment efficacy. Swift Meds directly targets and compresses this variable, thereby reducing negative health outcomes attributable to treatment delay.
3. **Economic Imperative:** The justification is rooted in hard economics. For patients, the system eliminates sunk costs from failed procurement trips. For pharmacies, it transforms invisible, stagnant inventory into a visible, monetizable asset. It attacks waste directly by providing the data necessary for predictive inventory management, directly reducing losses from expired stock.
4. **Technological Imperative:** The project is feasible and justified because the technological barriers have dissolved. The trifecta of real-time databases, ubiquitous geolocation, and secure payment APIs provides a mature, proven toolkit to solve this exact class of logistical problem. To not apply these tools is to accept known inefficiency.



II. OBJECTIVES OF THE PROJECT

A. GENERAL OBJECTIVE

The main objective of this project is to provide a platform who acts as an intermediary between the patients and the pharmacy, thereby enabling easy access to medication when needed.

B. SPECIFIC OBJECTIVES

- To design and develop a centralized, cloud-based web application that provides a unified platform for real-time medication search, reservation, and pharmacy management.
- To implement a real-time inventory tracking system by developing a secure API architecture that allows partner pharmacies to sync their stock levels automatically or update them manually with minimal effort, ensuring data accuracy of at least 98%.
- To integrate geolocation technology to enable users to locate the nearest available medication within a 10km diameter of their location and provide turn-by-turn navigation.
- To engineer a secure, multi-tiered user authentication and authorization system to manage distinct user roles (Clients, Pharmacists, Administrator) and protect sensitive personal and medical data.
- To develop a robust reservation and electronic payment gateway that allows users to reserve medications securely and allows pharmacies to confirm, prepare, and track orders, reducing the average acquisition time by over 90%.
- To conduct thorough unit, integration, and user acceptance testing (UAT) with a pilot group of at least 5 pharmacies and 100 users in Yaoundé to validate system functionality, usability, and reliability before full-scale deployment.



III. EXPRESSION OF NEEDS

A. FUNCTIONAL NEEDS

In this context, functional requirements describe what the system or application should do. The modules are as follows;

1. User Management

- The system shall allow users to register and create an account.
- The system shall authenticate users during login.
- The system shall allow users to view and edit their personal profiles.
- The system shall require admin approval for new pharmacy accounts before they can become active on the platform.
- The system shall allow admin to add, remove and activate/deactivate the accounts of all users.

2. Search and Discovery

- The system shall allow users to search for medications by name.
- The system shall display a list of pharmacies that have the searched medication in stock.
- The system shall display key details for each pharmacy (name, address, contact).
- The system shall allow users to filter results by proximity, price, and medication category.

3. Reservation and Payment

- The system shall allow authenticated users to reserve a specific quantity of a medication at a chosen pharmacy.
- The system shall reduce the pharmacy's inventory count upon a successful reservation.
- The system shall integrate a payment gateway to process secure online payments for reservations.

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



- The system shall send a confirmation notification to the user upon successful reservation and payment.

4. Pharmacy Management

- The system shall allow pharmacists to log in to a dedicated dashboard.
- The system shall allow pharmacists to add, update, and manage their inventory.
- The system shall provide pharmacists with a list of current and past reservations.
- The system shall allow pharmacists to update the status of a reservation (e.g., "Confirmed", "Ready", "Picked Up").
- The admin shall be able to verify crucial pharmacy details like location, contact information, and business license validity.

5. Analytics and Reporting

- The system shall provide the admin with a comprehensive dashboard showing key performance indicators (KPIs), such as:
 - ❖ Total number of users.
 - ❖ Total number of successful reservations.
 - ❖ Platform usage statistics.

6. Notification System

- The system shall automatically notify users about the status of their reservations.
- The system shall notify pharmacists of new reservations.

B. NON-FUNCTIONAL NEEDS

Non-functional requirements define the quality attributes of the system, rather than specific functions. They ensure the system operates effectively and efficiently.

1. Usability

- The user interface shall be intuitive and require minimal training to use.



- Critical tasks (e.g., searching for a medicine) must be completed in 5 clicks or less.

2. Performance

- The system shall load search results in under 5 seconds under normal load.
- The system shall support at least 50 concurrent users without significant performance degradation.

3. Reliability and Availability

- The system shall be available 99.0% of the time.
- The system shall have mechanisms to handle errors gracefully and prevent data loss.

4. Security

- All user passwords shall be encrypted (hashed and salted) in the database.
- All data transmissions shall be secured using HTTPS/SSL protocols.
- The system shall implement role-based access control to ensure users can only access authorized functions and data.

5. Scalability

- The system architecture shall be designed to allow for easy scaling to accommodate more users and pharmacies in the future (e.g., to other cities).

6. Maintainability

- The source code shall be well-documented and structured to allow for easy updates and bug fixes by new developers.

IV. PROJECT PLANNING

A. Chronogram of Activities

Table 4: Chronogram of Activities

PHASE	OBJECTIVE	OUTPUT	DURATION	PERIOD
INSERTION	Collection of information on the enterprise	Insertion Book	2 weeks	1 st July to 13 th July
EXISTING SYSTEM	Study of the existing system	Existing System	1 week	14 th July to 20 th July
SPECIFICATION BOOK	Specification of the user needs	Specification Book	1 week	21 st July to 27 th July
ANALYSIS	Capture of needs Use case and textual description Modeling	Analysis Book	2 weeks	28 th July to 10 th August
CONCEPTION	Preliminary conception and Detailed conception	Conception Book	2 weeks	11 th August to 24 th August
REALIZATION	Implementation Unitary test Integration Test Development	Realization Book	3 weeks	25 th August to 14 th September
TEST OF FUNCTIONALITIES	Testing of the software and debugging	Test of functionalities	1 week	15 th September to 21 st September
INSTALLATION AND USER GUIDE	Documenting software	User Guide	1 week	22 th September to 30 th September

B. Gantt Project Planning

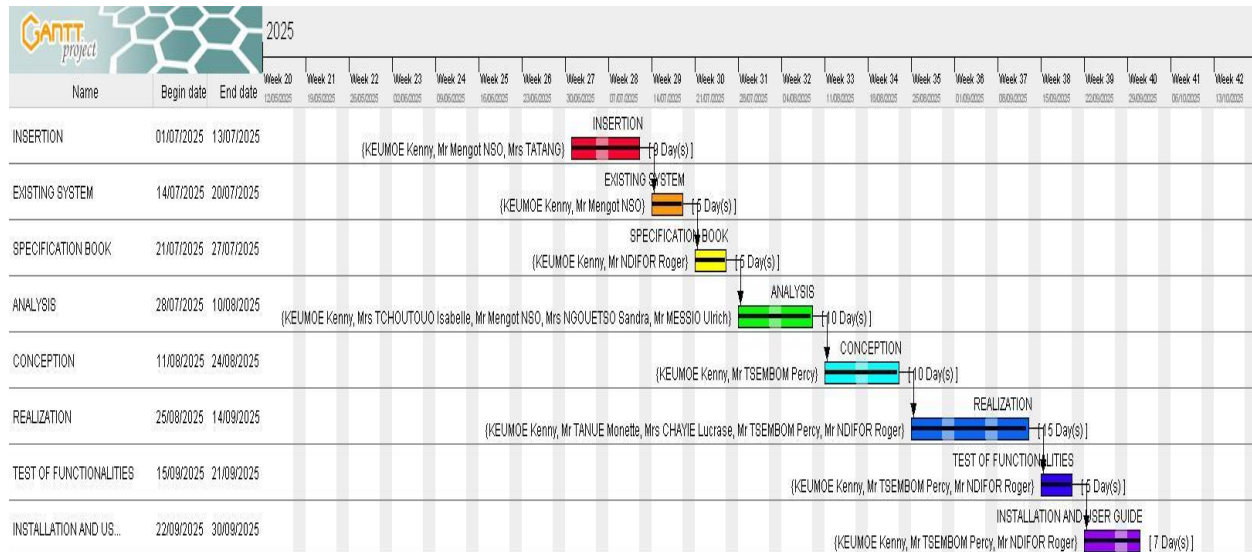


Figure 3: Project Gantt chart

V. ESTIMATED COST OF THE PROJECT

A. Software Resources

Table 5: Software Resources

RESOURCES	DESIGNATION	USAGE	QUANTITY	UNIT COST (FCFA)
Formatting	Microsoft Office Professional 2019	Writing of the report	1	201250
Web browser	Firefox	View web pages	1	Freeware
Code Editor	Visual Studio Code	For writing the code of the application	1	Freeware
Project Planning	Gantt Project	For building a Gantt chart	1	Freemium
Graphic Editor	Icogram	For building a geographical location of the enterprise	1	Freemium
UML Analysis	Visual Paradigm	For drawing UML diagrams	1	Community Version
Version Control	Git and GitHub	For tracking and managing changes to software code	1	Free Version
Hosting Platform	Vercel	For the deployment of the frontend and backend servers of our system.	1	Free Tier

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

Hosting Platform	Supabase	For the hosting of the database.	1	Free Tier
TOTAL 1			9	201250

B. Hardware Resources

Table 6: Hardware Resources (source: mercurial.cm)

RESOURCES	HARDWARE	QUANTITY	UNIT COST (FCFA)
Computer	HP EliteBook 8540w intel core i7 vPro 500GB SDD	1	250000
Printer	HP OFFICEJET PRO	1	200600
Network	Local network installation	1	300000
Removable Storage	USB HP Flash Drive	1	13800
Smartphone	Tecno Pop 7	1	67841
TOTAL 2		5	832241

C. Human Resources

Table 7: Human Resources (source: Mercurial 2025)

ROLE	NUMBER OF DAYS	QUANTITY	COST PER DAY	TOTAL PRICE (FCFA)
Project Manager	40	1	30000	1200000
Analyst	20	1	25000	500000
UI/UX Designer	15	1	10000	150000
Programmer	40	1	15000	600000

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

Tester	20	1	10000	200000
Margin error	/	/	/	1635000
TOTAL 3				3250000

D. Global Estimation

Table 8: Global Estimation

TOTAL 1 (FCFA)	TOTAL 2 (FCFA)	TOTAL 3 (FCFA)	TOTAL SUM (FCFA)	CONTINGENCY RESERVE (10%)	OVERALL TOTAL (FCFA)
201,250	832,241	3,250,000	4,283,491	428,349	4,711,840
FOUR MILLION SEVEN HUNDRED ELEVEN THOUSAND EIGHT HUNDRED FORTY					



VI. PROJECT CONSTRAINTS

A. Technical Constraint

For the development of our system, we possess robust tools that ensure minimal level of security, extensibility, and excellent scalability. Additionally, the programming phase will have to follow all the technical standards for a better performance in a reduced execution time; this is why the choice of the development technologies is crucial.

B. Time Constraint

The project will be completed within a period of 12 weeks (3 months), commencing on the start date of July 01st, 2025.

C. Cost Constraint

The implementation of our project will necessitate expenses on human, materials, and software, amounting to a total of **4,711,840FCFA**.

VII. LIST OF PARTICIPANTS AND DELIVERABLES

A. LIST OF PARTICIPANTS

Table 9: List of Participants

NAME	FUNCTION	ROLE
Mr. TSEMBOM Percy Linkwe	Follows up interns at the company level	Professional Supervisor
Mr. NDIFOR Roger	Follows up student at the academic level	Academic Supervisor
KEUMOE KENNY LOANN	AICS Student Intern	Student at AICS

B. DELIVERABLES

In project management, any component materializing the result of a realization service is called a deliverable. In the case of our project, the deliverables are:

- The Application
- User Guide
- PowerPoint



CONCLUSION

As we conclude this section, we have clearly outlined the objectives to be achieved. The specification document has allowed us to identify the various stakeholders involved in the project, along with the necessary requirements and preliminary planning for its success. We will now transition to the next phase, which is the analysis stage. During this phase, we will model our system using a modeling language and a unified process, as well as conduct a comparative study of UML and Merise.



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



FILE 3: ANALYSIS BOOK

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



PREAMBLE

The main objective of the analysis phase is to capture the user's need, the delimitation of the field of study and to have a clear understanding of the system in study. To achieve this, we will use UML (Unified Modelling Language) with the 2TUP (2 Track Unified Process) as method applied to UML to analyze the system. We will start by doing a comparison between UML and MERISE, of various unified processes and lastly, we will present the modelling of the solution we propose.

CONTENT

INTRODUCTION

- I. COMPARATIVE STUDY OF UML AND MERISE
- II. COMPARATIVE STUDY OF UNIFIED PROCESS
- III. CHOICE OF THE ANALYSIS METHOD
- IV. MODELLING OF THE PROPOSED SOLUTION

CONCLUSION



INTRODUCTION

System development can be viewed as comprising two main components: system analysis and system design. Both are essential for understanding the intricacies of the existing system or the new system being developed. Typically, the analysis and design of information systems aim to facilitate the creation of databases that accurately represent the realities of the studied domain, necessitating the use of a structured design method. Consequently, we have chosen the UML method, as it provides significant advantages for developers seeking a user-centered approach and broad design capabilities. This section of the report will include a comparative study of UML and MERISE an exploration of unified processes, and an overview of various diagrams that address functional requirements.

I. COMPARATIVE STUDY OF UML AND MERISE

A. MERISE

MERISE stands for “Méthode d’Etude et de Réalisation Informatique pour des Systèmes d’Entreprise”. Although it is prescriptive to some extent, MERISE permits the participation of end users and senior management as well as data processing professionals in its decision cycle. MERISE is a method for designing, developing and carrying out IT projects. The goal of this method is to achieve the design of an information system. The MERISE method is based on the separation of data and processing to be carried out in several conceptual and physical models. The essentials of the approach lie in its three cycles: the decision cycle, the life cycle and the abstraction cycle, which cover data and process elements equally. The separation of data and processing ensures longevity in model. Indeed, the arrangement of data does not have to be often overhauled, while treatments are more frequently.

B. UML

UML (Unified Modelling Language) is a standard notation for the modelling of real-world objects as a first step in developing an object-oriented design methodology. Its notation is derived from and unifies the notations of three object-oriented design and analysis methodologies: Grady Booch's methodology for describing a set of objects and their relationships, James Rumbaugh's Object-Modelling Technique, Ivar Jacobson's approach which includes a use case methodology. Other ideas also contributed to UML, which was the result of a work effort by Booch, Rumbaugh, Jacobson, and others to combine their ideas, working under the sponsorship of Rational Software. UML captures information about the static and dynamic view of a system. UML 2.5 comprises of 14 diagrams which represent the different views of a system. The 14 diagrams can be subdivided into two, Static or structural and Dynamic diagrams. These diagrams include;

1. STATIC OR STRUCTURAL DIAGRAMS

- Class diagram
- Object diagram

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



- Package diagram
- Component diagram
- Deployment diagram
- Composite Structure diagram
- Profile diagram

2. BEHAVIORAL OR DYNAMIC DIAGRAMS

- Use case diagram
- Activity diagram
- Sequence diagram
- Communication diagram
- State Machine diagram
- Global Interaction diagram
- Timing diagram

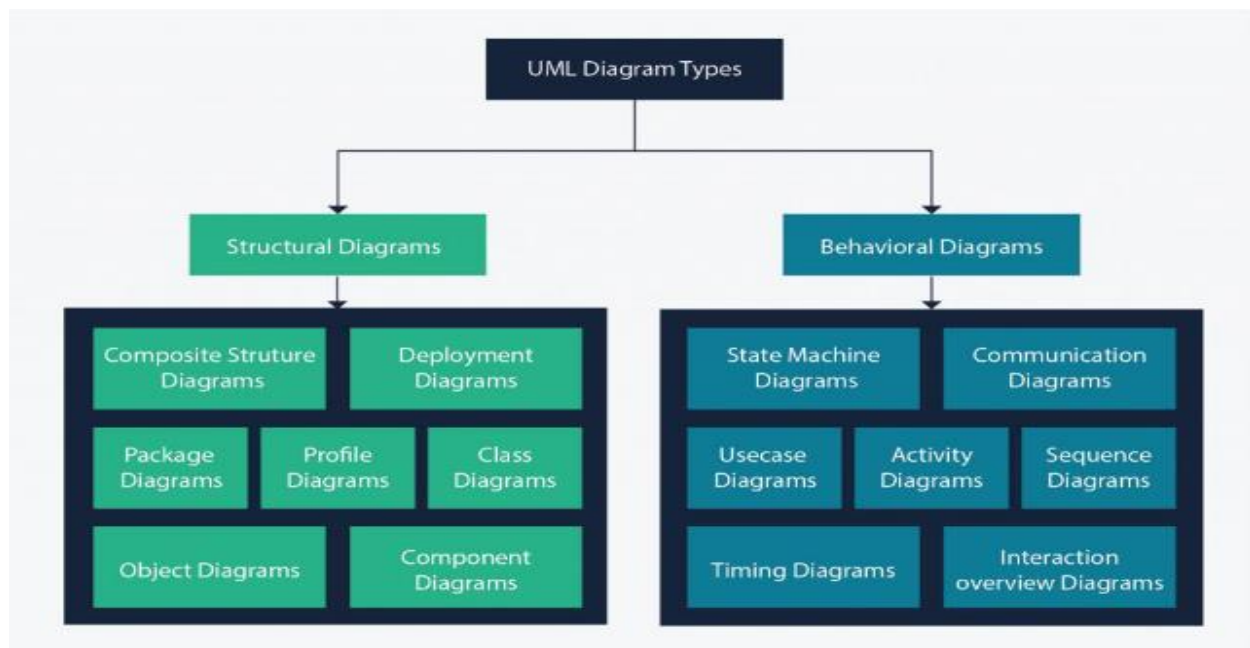


Figure 4: UML 2.5 diagrams overview (source: <https://d3n817fwly711g.cloudfront.net/uploads/2012/02/UML-Diagram-types-1-1024x658.png>)

On very important notice is that UML is not a method but a modelling language. As such to give it an approach we need to associate UML to a Unified Process (UP) in other to give our conception a methodology to follow. There exist several Unified Processes but our modelling approach will be the 2TUP (Two-track unified process) which we will use in the course of our project.

Table 10: Comparison between MERISE and UML

UML	MERISE
UML is however not a method but rather an object modelling language to which it is necessary to associate an approach to make it a method. This is the case with the 2TUP method; RUP and XP.	MERISE is a systemic method of analysis and design of information systems. That is, it uses a system approach
Emphasizes system behavior and structure using 14 diagram types, such as use case diagrams, class diagrams, and activity diagrams	Emphasizes data analysis and modelling using data flow diagrams (DFDs), entity-relationship diagrams (ERDs), and data dictionaries
Suitable for object-oriented design and implementation	Suitable for relational databases design and implementation

II. COMPARATIVE STUDY OF UNIFIED PROCESS

A. Unified Process

A Unified Process is a process of development of software constructed on UML; it is iterative, incremental, centered on architecture, driven by use cases and requirements.

Iteration is distinct sequence of activities with a basic plan and evaluation criterion that produces an internal or external output. Either the content of iteration is improved, or the evolution of the system is evaluated by users.

An increment is the difference between two released products at the end of two iterations. Each iteration that the group is capable of integrating the technical environment in order to develop a final product and give users the possibility of having tangible results.

Centered on architecture the different models derived during the establishment of system must be reliable and coherent.

Driven by use case and requirements enables the clear definition of users' needs and priorities respectively thereby minimizing the risk of project failure.

B. Two Track Unified Process (2TUP)

2TUP is a unified process which is built on UML and has as objective to bring solution to constraints of functional and technical changes imposed on information systems by strengthening controls on development capacities. It proposes a Y-sharped development life cycle that separates the functional aspect from the technical aspects, and the merging of these two forms the implementation aspect. 2TUP distinguishes therefore two branches: the functional and technical branches, the combination of the result of these two branches forms the third: the realization branch where we realize our system. The diagram below illustrates the branches of 2TUP.

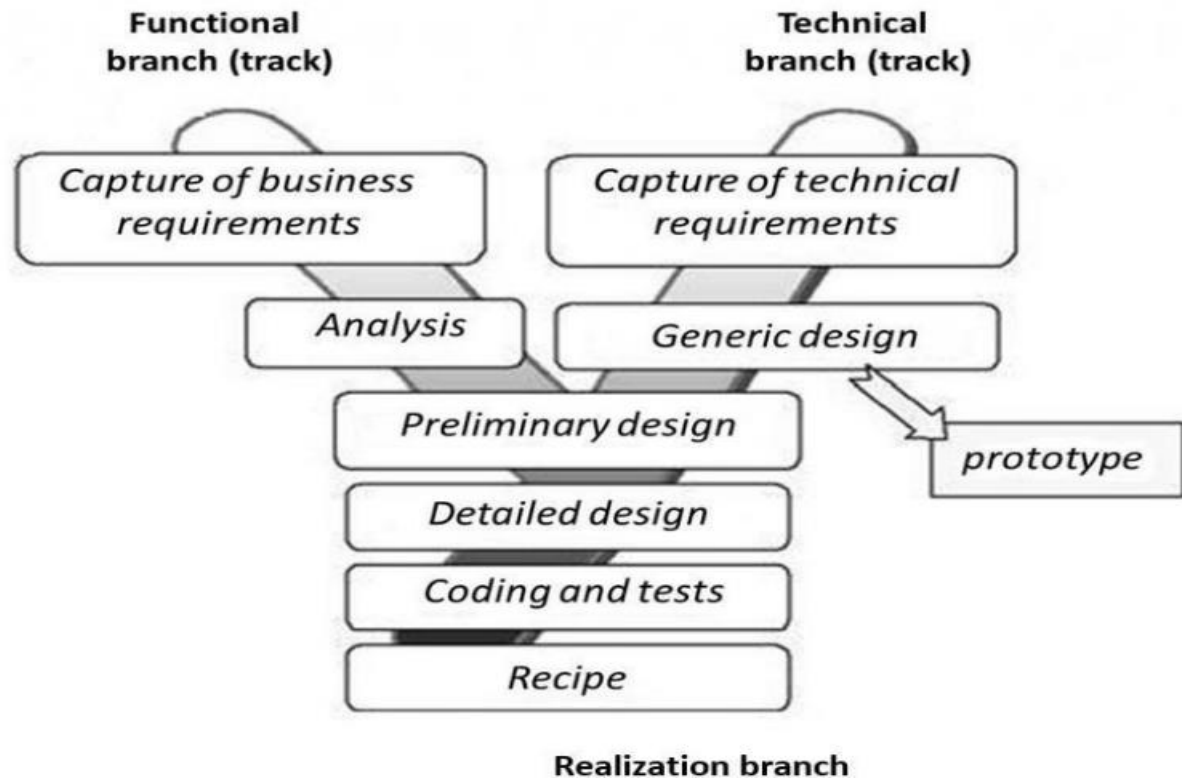


Figure 5: Two Track Unified Process (source: <https://www.mysciencework.com/omniscience/pervasive-mobile-healthcare-system-basedon-cloud-computing>)

1. The Left Branch (Functional branch)

It captures the functional needs of a system. This ensures the production of software that meets the needs/requirements of the user. The analysis here consists of studying precisely the functional specification in order to obtain an idea of what the system is going to realize, and its result does not depend on any technology.

2. The Right Branch (Technical branch)

The technical branch enumerates the technical needs and proposes a generic design validated by a prototype. The technical needs include constraints and choices related to the conception of the system, the tools and equipment as well as the integration constraint with the existing system condition.

3. The Middle Branch (Realization or Implementation branch)

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



In this branch, we study the preliminary conception, detailed conception, and documentation of the system. The realization branch supports the following:

Preliminary conception: This is the most sensitive step of 2TUP as it is the confluence of the functional and technical branch. It is completed when the deployment model, the operating model, the logical model, interfaces and the software configuration model are defined. We have the following diagrams:

- Component Diagram
- Deployment Diagram
- Package Diagram
- Composite Structure Diagram

Detailed conception: This is the detailed design of each feature of the system. We have the following diagrams:

- Class Diagram
- Object Diagram
- Sequence Diagram
- Timing Diagram

Coding and testing: This is the phase where we program the designed features and test the coded features.

The recipe: Also known as the deliverables is the validation phase of the functions of the developed system.



III. CHOICE OF THE ANALYSIS METHOD

The reason why we chose UML modelling language and the software development process 2TUP instead of many others that exist, include:

- UML is the current standard for programming in an object-oriented language. For this reason, it is widely understood and well known making it easy for a new programmer to join the project and be productive from the very first day.
- UML diagrams allow teams to virtualize how a project is or will be working, and they can be used in any field, not just software engineering. The diagrams will allow teams to virtualize together how a system or a process will work or did work. It can provide new ideas for how teams have to collaborate to achieve the goal of the workflow process.
- 2TUP is centered on the creation and maintenance of a model, rather than the production of mountain documents.
- 2TUP is user oriented as it permits the development of software that responds to the needs of the users through the study of the user needs.
- 2TUP is iterative and incremental, hence it enables the project team to produce refined amelioration if necessary and easily integrate it in the already existing system.
- 2TUP by permitting the project team identify and test the key functionalities of the system limits the risk related to building the system.

IV. MODELLING THE PROPOSED SOLUTION

A. Capture of the Functional Needs

The first step of the left (functional) branch of Two Track Unified Process (2TUP) is the capture of the functional needs. At this step, we capture the intended behavior of the system that maybe express as services, tasks or functions the system is required to perform.

1. USE CASE DIAGRAM

i. Definition

Use case diagram shows the functionalities of a system, their inter dependencies and how they relate with actors of the system. A use case is a specification of behavior. The main objectives of the use case diagram are:

- Provide a high-level view of the system.
- Identify the functions of the system.

Use case diagrams are completed with a textual description of each use case that is intended to define the use case in greater details

ii. Formalism

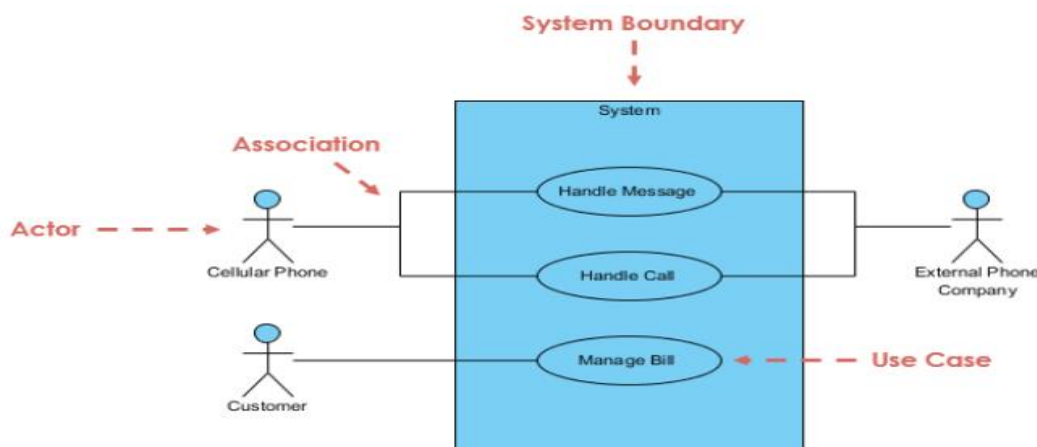
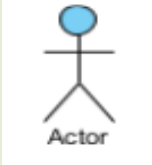
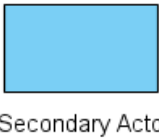


Figure 6: Use case diagram formalism

iii. Components of a Use case diagram

Table 11: Component of a Use case diagram

Element	Notation	Description
Actor		Represents an entity that directly interacts with the system. The actor is what performs the different possible actions of the system
Secondary actor		
Use case		A use case represents a functionality of the system. It is an action that can be performed by an actor.
Association		It indicates that an actor takes part in a use Case
Include		An inclusion denotes that an included action must be performed before the including action can be performed.
Extend		An extension denotes that an extending action may be performed while an extended action is being performed.
Generalization		This shows that an actor or a use case is a kind of another abstract or concrete actor which can be

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

		defined and later specialized using generalization relationship.
System		It is a container of use cases which interact with external actors

iv. Primary actors of our system

Table 12: Primary actors of our system

Primary actors	Role
Visitor	To create an account in the system.
Client	To find and secure required medication as quickly and easily as possible.
Pharmacist	To manage pharmacy operations efficiently and serve more customers through the platform.
Admin	To ensure the platform runs smoothly, securely, and fairly for all users.

v. Secondary actors of our system

Table 13: Secondary actors of our system

Secondary actors	Role
Google Map API	To provide geolocation and mapping services.
Campay API	To process financial transactions securely and reliably.

vi. Swift Meds use case diagram

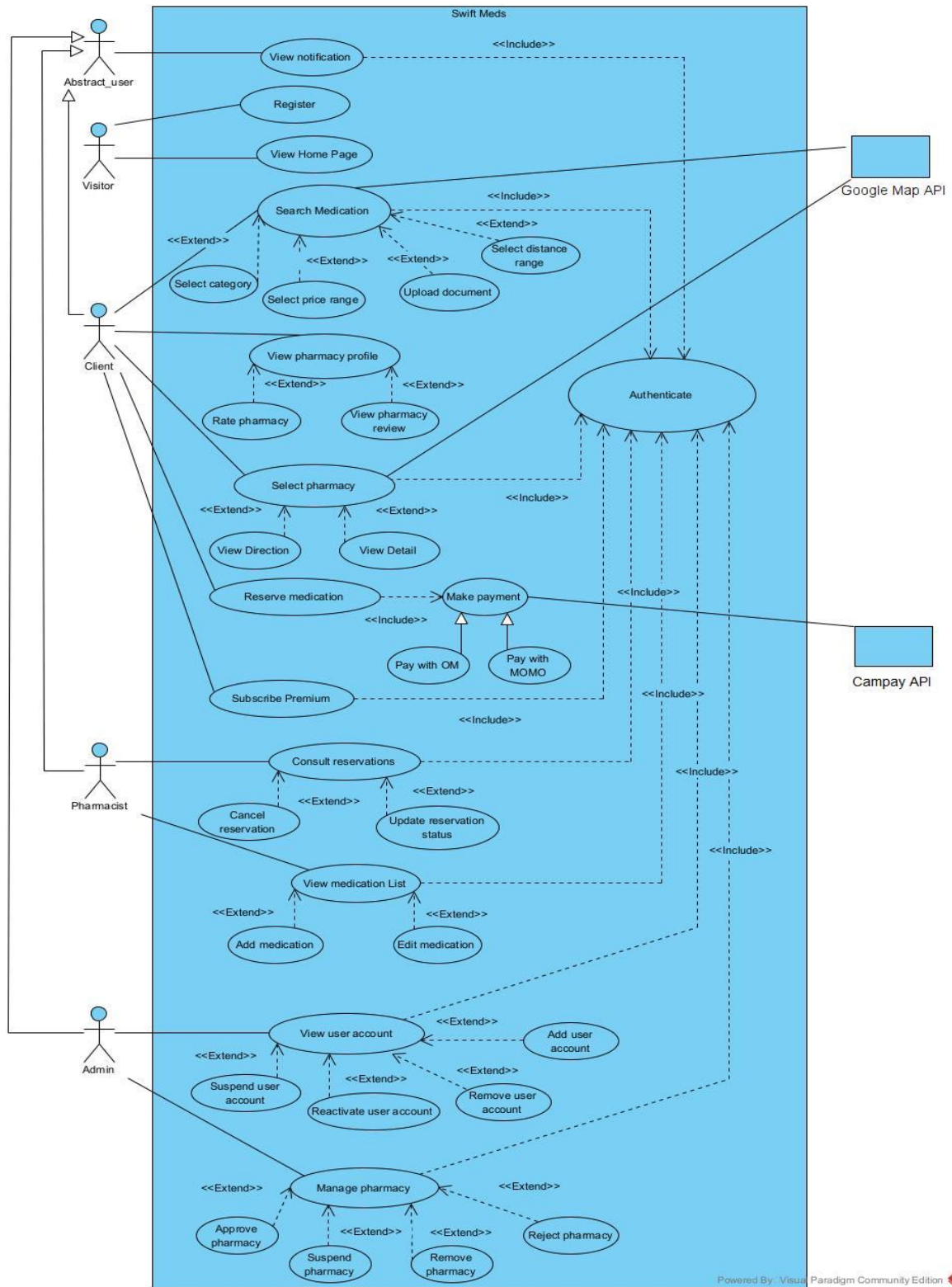


Figure 7: General use case diagram

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

2. Textual Description

i. Definition

UML allows the execution of a use case to be described in a textual way, in a form called nominal scenario. A nominal scenario describes in more detail the execution of a use case by an actor until it is successfully completed.

ii. Formalism

A textual description of a use case is represented in the following form

Table 14: Textual description formalism

Title	Name of Use case
Summary	Aim of the use case
Actors	Users
Pre-condition	Aim of the use case Set of actions that must be completed before the launching of the use case
Trigger	The event that causes the use case to be initiated
Nominal Scenario	It corresponds to a normal development of a use case
Alternative Scenario	The variants of a nominal scenario
Post Condition	Set of mechanism that can lead to the end of the use case



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

3. ACTIVITY DIAGRAM

i. Definition

An activity diagram is a graphical representation of workflows that show the steps needed in the realization of a process; showing the details from a start point to an end point through all decisions and actions that can possibly be performed. Activity diagrams are intended to model both the computational and organizational process. They flow can be sequential, branched or concurrent. Below is the activity diagram formalism.

ii. Formalism

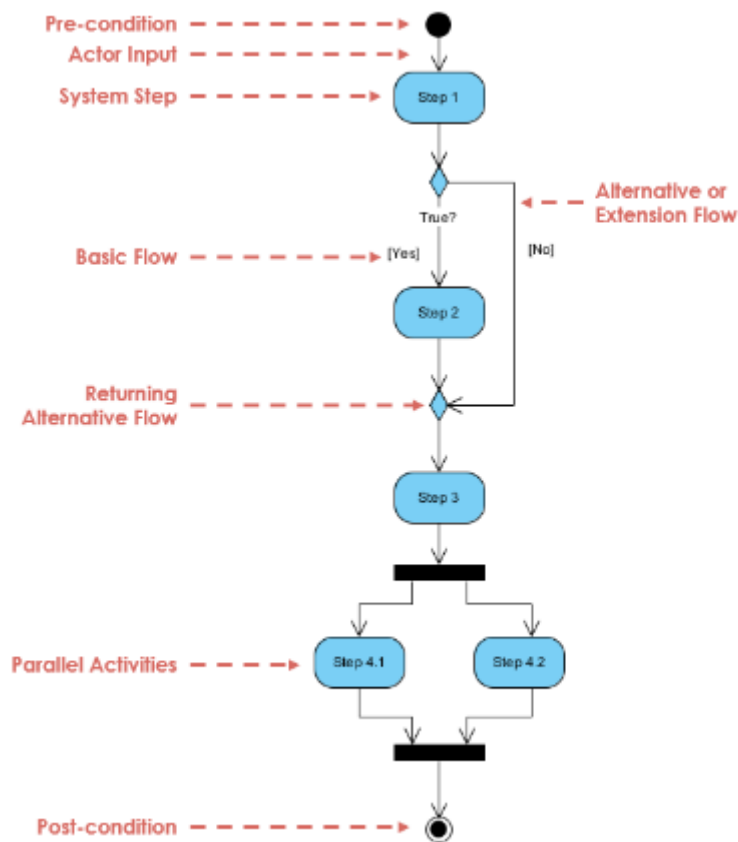
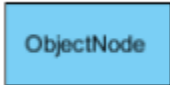
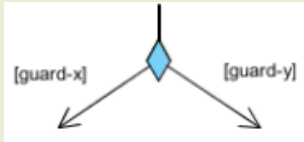


Figure 8: Activity diagram formalism

iii. Component of an Activity diagram

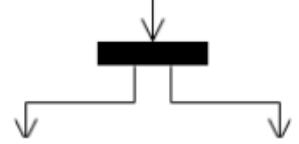
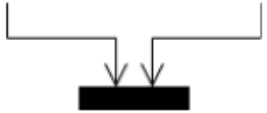
Table 15: Component of an Activity diagram

Element	Notation	Description
Activity		Used to represent a set of actions.
Action		Represent a task to be performed.
Activity Edge		A directed connection between two activity nodes.
Initial node		Shows the beginning of an activity or set of actions.
Final node		Stop all control flows and object flows in an activity
Object node		Represent an object that is connected to a set of Object Flows.
Decision node		Represents a test condition that splits an incoming activity edge into opposite outgoing activity edges.
Merge node		Brings back together different decision paths created using a decision node.
Fork node		Slits behavior into parallel or

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



		concurrent flows of activities (or actions).
Join node		Bring back together a set of parallel or concurrent flows of activities (or actions).
Swimlane and Partition		A way to group activities performed by the same actor on an activity diagram or to group activities in a single thread

iv. Activity Diagram of Authenticate

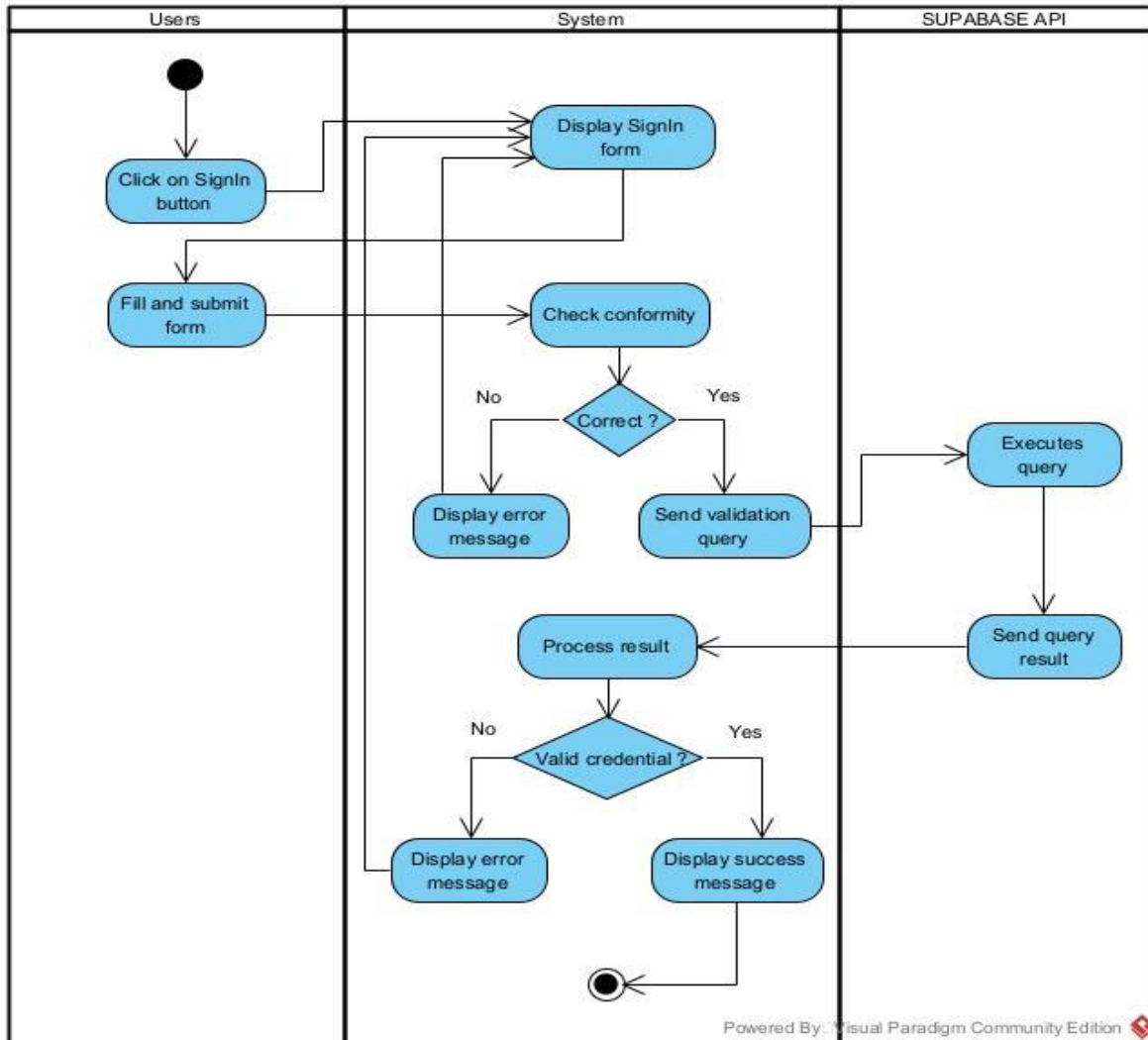


Figure 9: Authenticate activity diagram

v. Activity Diagram of Reserve Medication

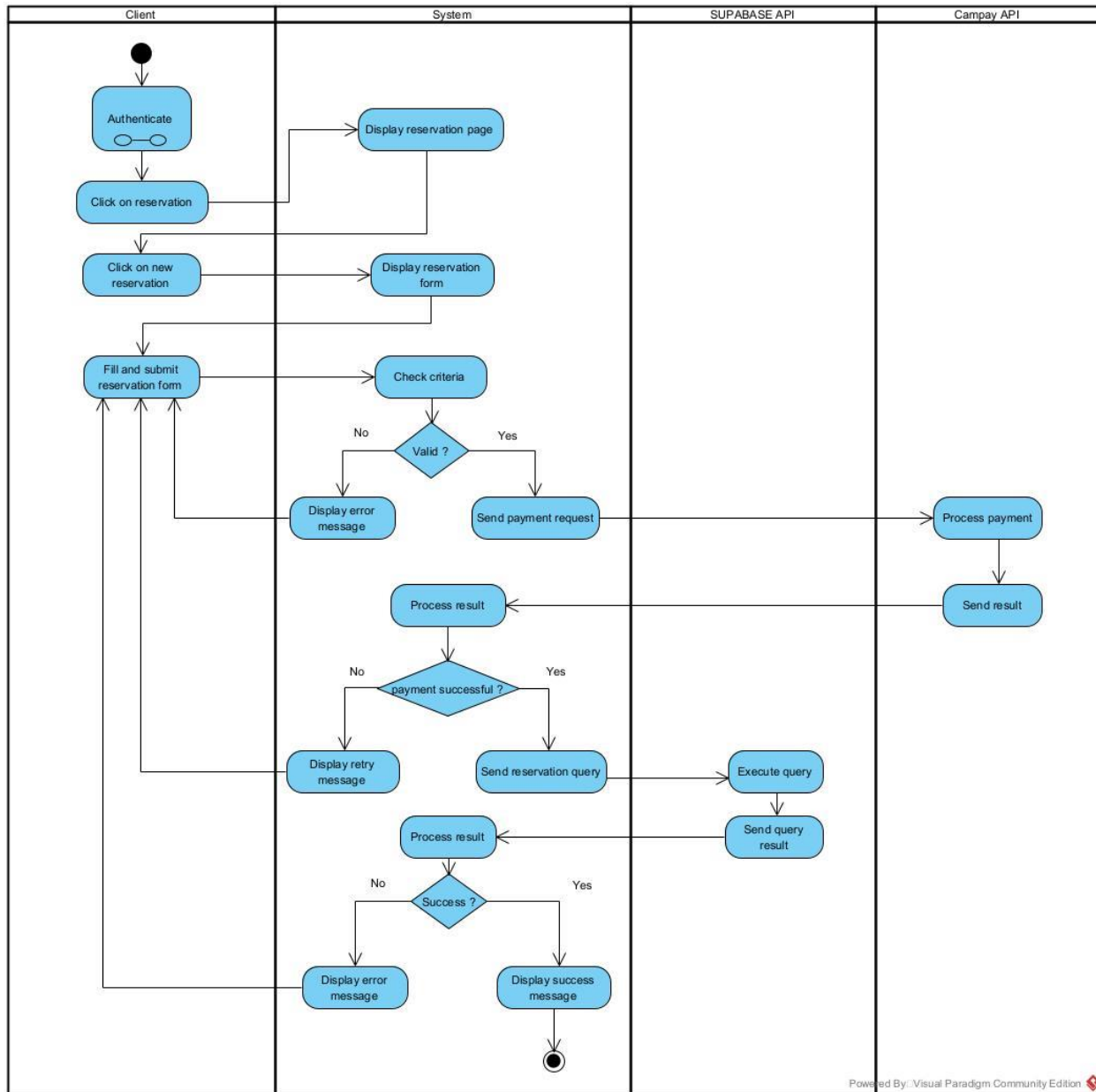


Figure 10: Reserve Medication activity diagram

vi. Activity Diagram of Search Medication

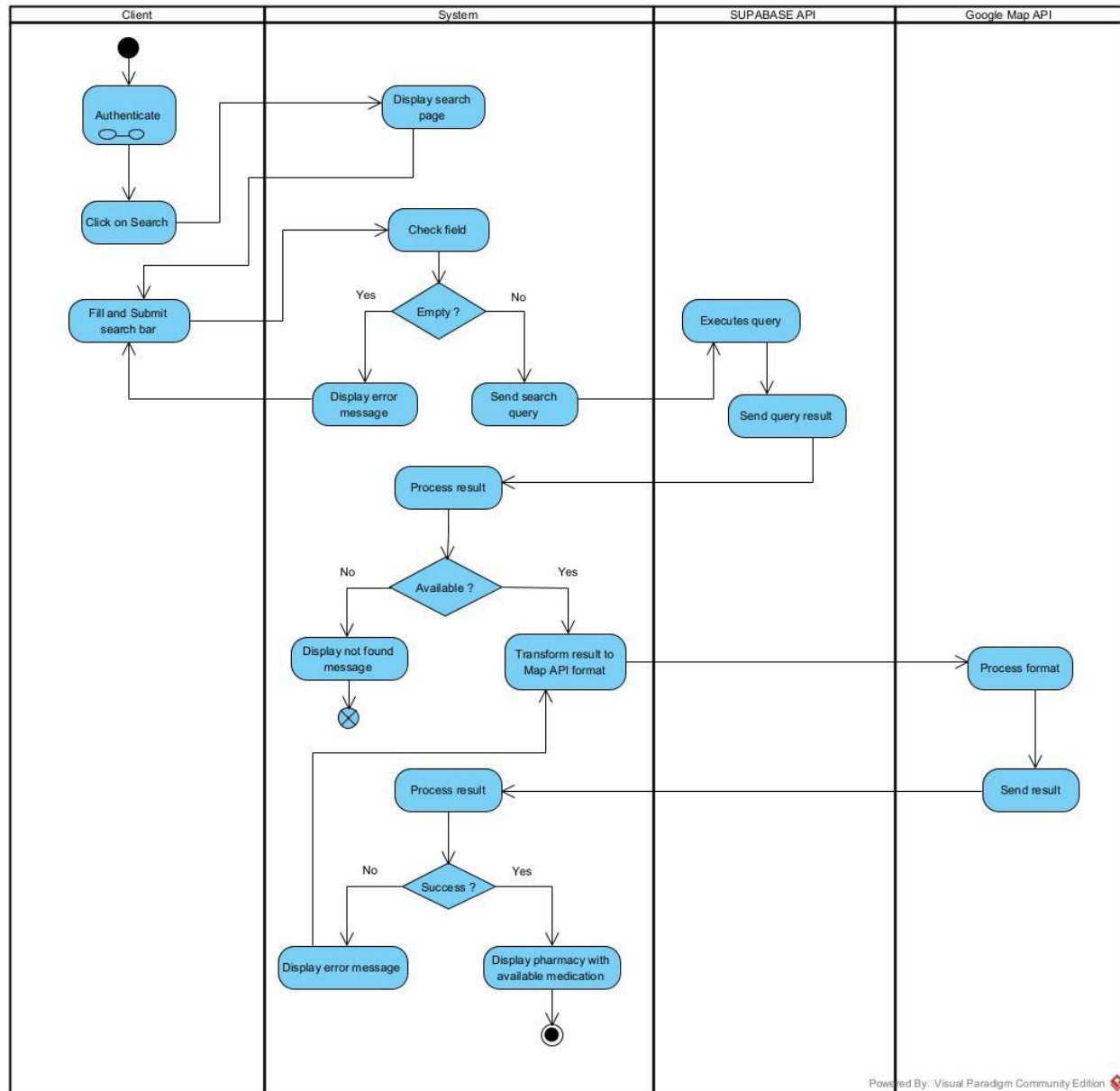


Figure 11: Search Medication activity diagram

4. SEQUENCE DIAGRAM

i. Definition

A Sequence diagram describes interactions among classes in terms of an exchange of messages over time. They are also called event diagrams. A Sequence diagram is a good way to visualize and validate various runtime scenarios. These can help to predict how a system will behave and to discover responsibilities a class may need to have in the process of modelling a new system.

ii. Formalism

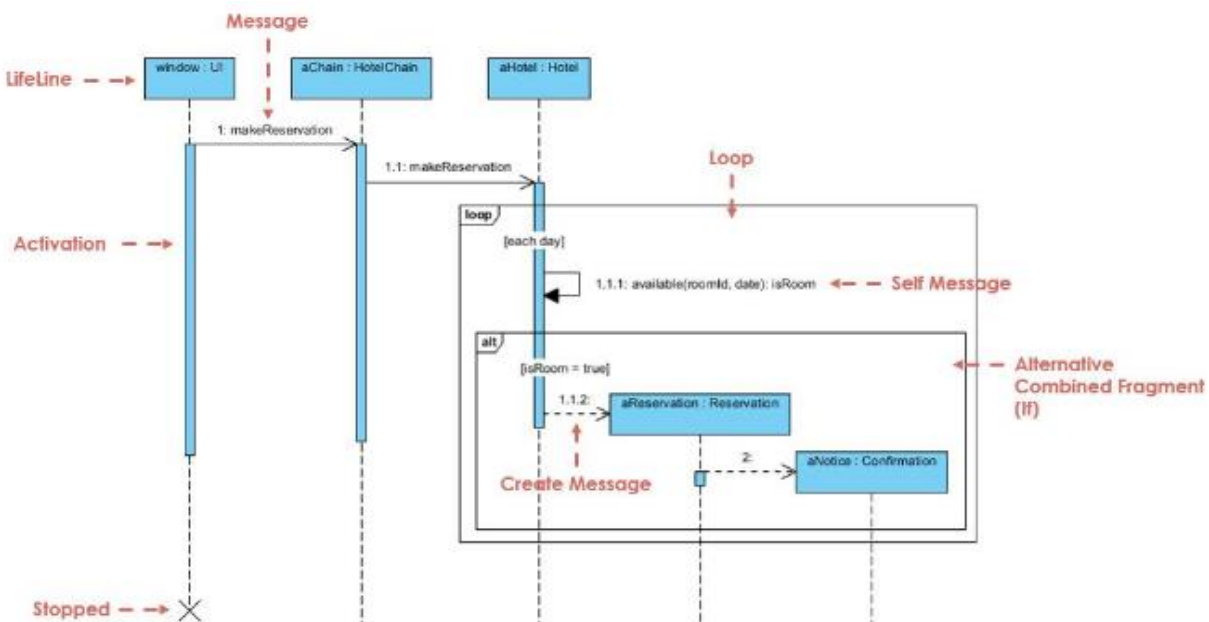


Figure 12: Sequence diagram formalism

iii. Component of a Sequence diagram

Table 16: Component of sequence diagram

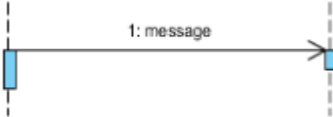
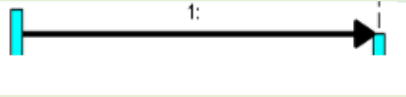
Element	Notation	Description
Lifelines		They represent rows or objects instances that participate in the sequence being modelled.
Asynchronous Message		It is a message that receives an indirect response.
Synchronous Message		It is a message that sends and want response before it continues a process
Self-Message		Self-message is a kind of message that represents the invocation of message of the same lifeline.
Return Message		It represents the response of a message.
Actor		They send and receive message.
Combined Interaction Fragment		An articulation of interaction diagram, defined by an operator and operands.
Object		They send and receive messages.

Figure 13

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

iv. Sequence Diagram of Authenticate

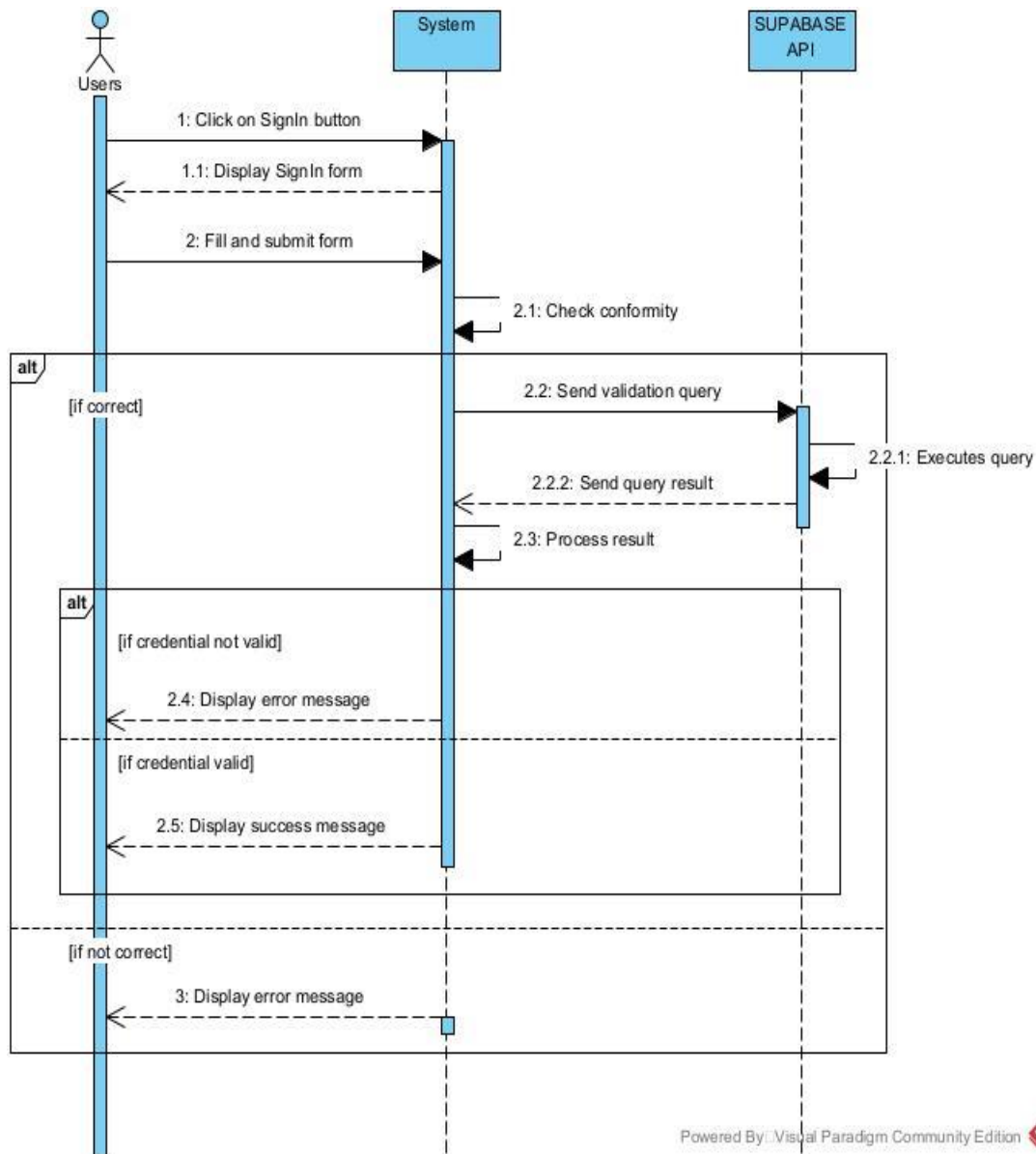


Figure 14: Authenticate sequence diagram

v. Sequence Diagram of Reserve Medication

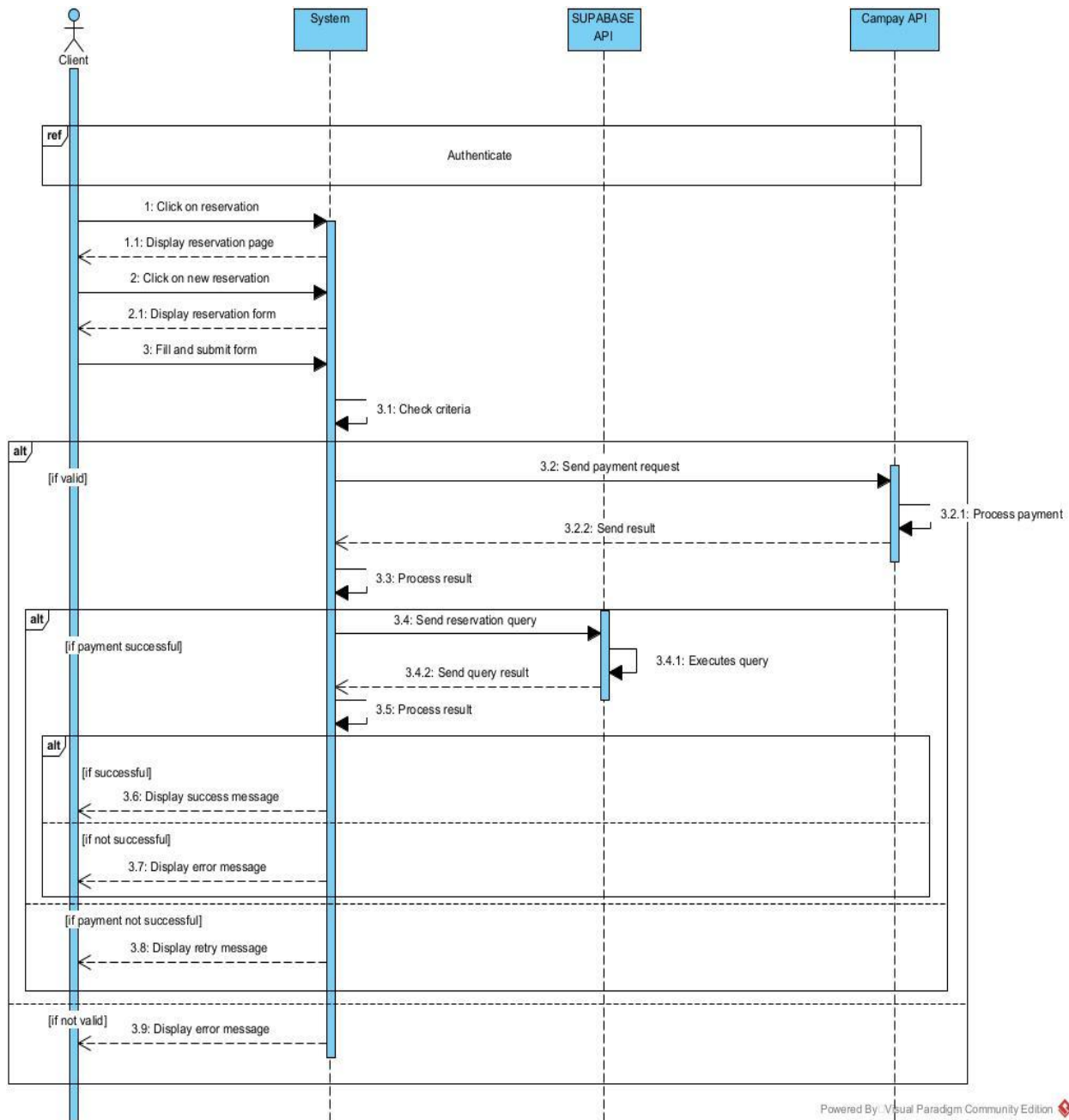


Figure 15: Reserve medication sequence diagram

vi. Sequence Diagram of Search Medication

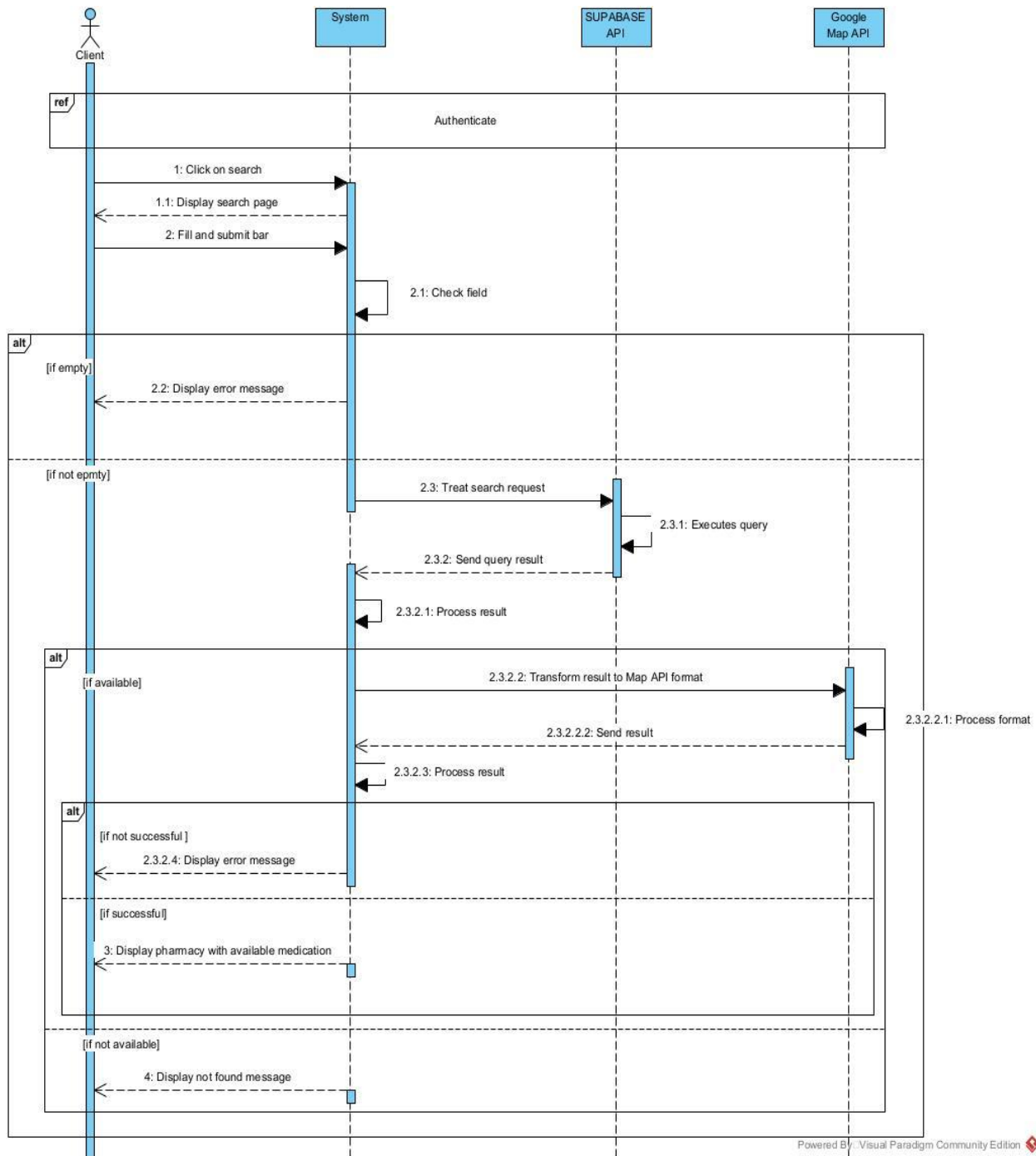


Figure 16: Search medication sequence diagram

5. COMMUNICATION DIAGRAM

i. Definition

Communication Diagrams model the interactions between objects in a sequence. They describe both the static structure and the dynamic behavior of a system. It is a simplified version of a Collaboration Diagram introduced in UML 2.0. A communication diagram is more focused on showing the collaboration of objects rather than the time sequence.

ii. Formalism

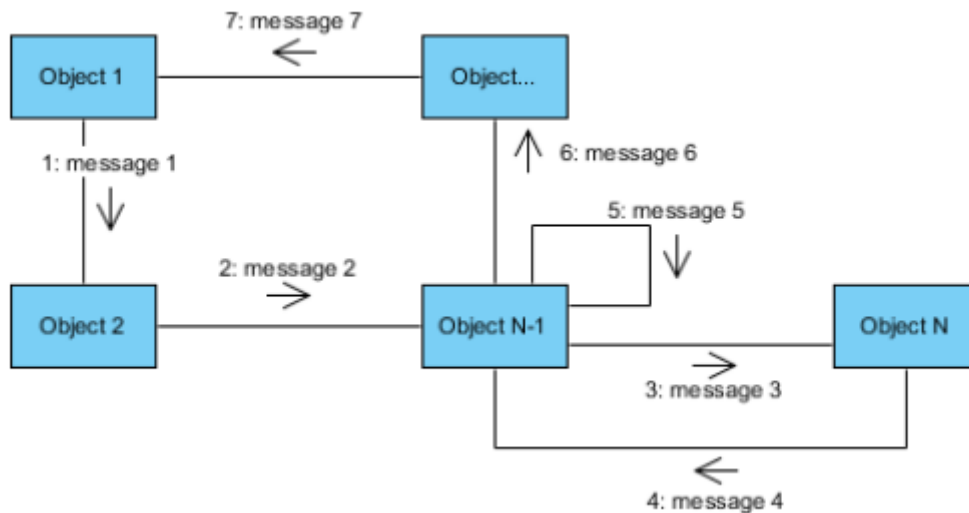
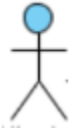


Figure 17: Communication diagram formalism

iii. Component of a Communication diagram

Table 17: Component of a Communication diagram

Element	Notation	Description
Message		Designs a particular communication between lifelines.
Actor		A role play by an entity that interacts with the subjects
Link		It initiates an association it connects two objects together for them to communicate.
Object		An object represents an individual participant in the interaction conversation.

iv. Communication Diagram of Authenticate

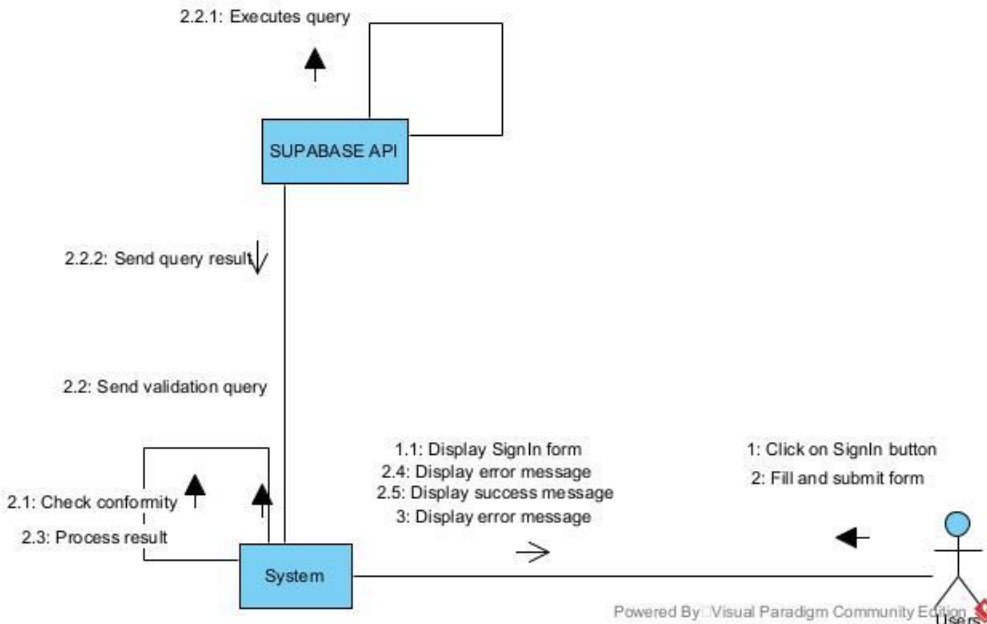


Figure 18: Communication diagram of Authenticate

v. Communication Diagram of Reserve Medication

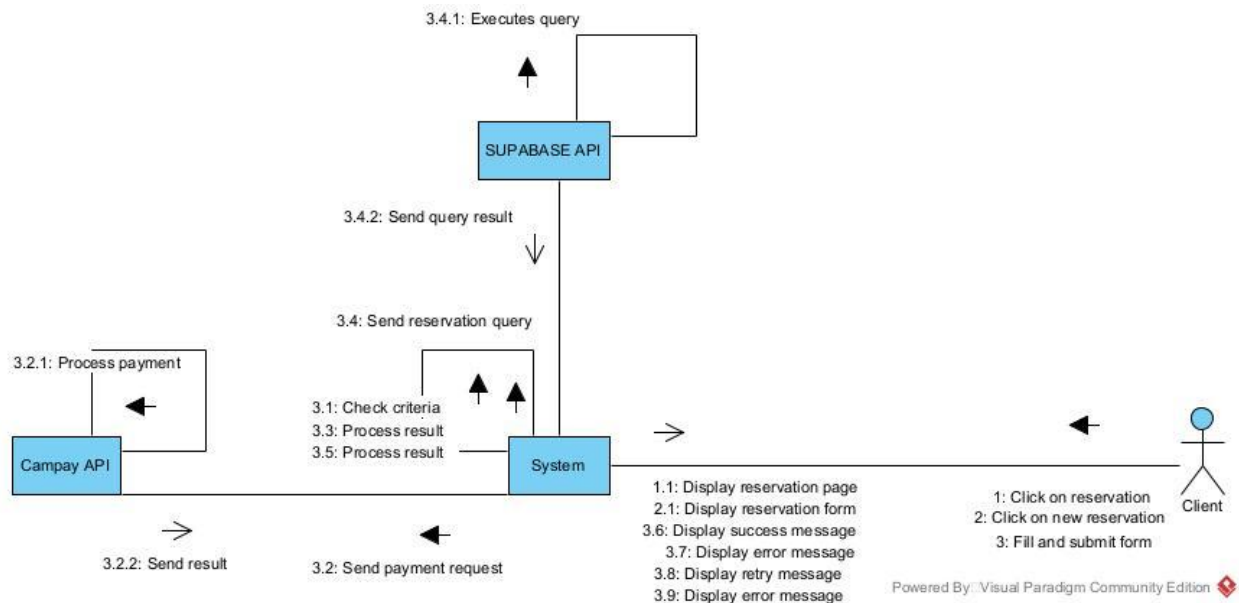


Figure 19: Communication diagram of Reserve Medication

vi. Communication Diagram of Search Medication

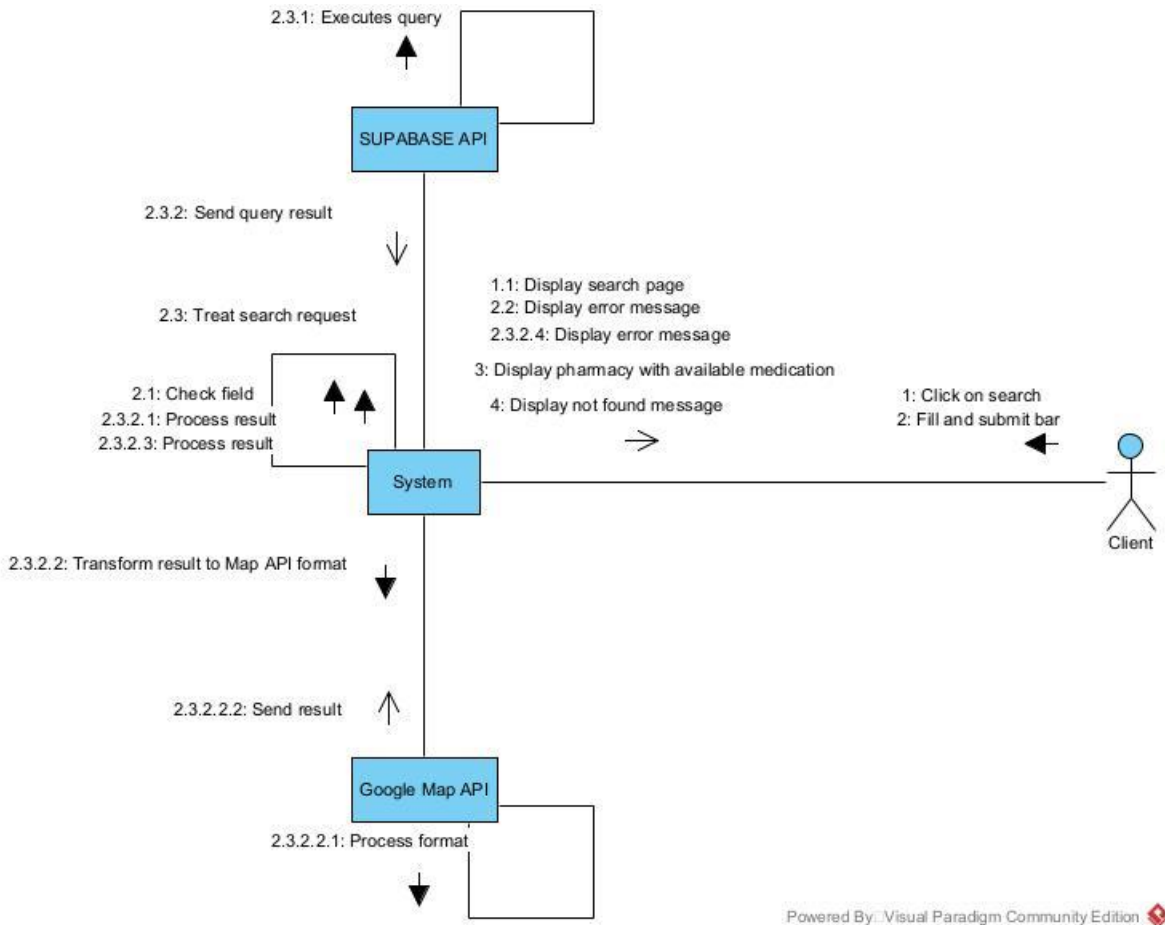


Figure 20: Communication diagram of Search Medication



CONCLUSION

In the analysis phase, we selected a software development process and a modeling language, after which we outlined the functional requirements of our system. We examined the use case diagram, which illustrates the relationships between the actors and the use cases (the actions the actors can perform within the system). We also explored the communication diagram, which depicts the system's architecture based on object-oriented programming principles. Additionally, we analyzed the sequence diagram, which outlines the flow of messages between elements in the system. Finally, we reviewed the activity diagram, which illustrates the workflow of our system. We will now transition to the conception phase, where we will present the technical aspects of our system along with the associated diagrams.



FILE 4: CONCEPTION PHASE



PREAMBLE

The conception phase will permit us to present in an orderly manner the components necessary for the good functioning of our software and also the architecture used for the proposed solution. It bridges the gap between the analysis phase and the realization phase.

CONTENT

INTRODUCTION

I. TECHNICAL BRANCH

A. GENERIC BRANCH

B. CAPTURE OF TECHNICAL NEEDS

C. RELATED UML DIAGRAMS

1. CLASS DIAGRAM
2. OBJECT DIAGRAM
3. STATE MACHINE DIAGRAM
4. PACKAGE DIAGRAM

CONCLUSION



INTRODUCTION

The conceptual phase will provide a detailed description of the necessary specifications, features, and operations required to meet the functional requirements of the proposed system, as outlined in the analysis phase. This stage aims to identify and consider essential components hardware and software as well as the system's structure (including network capabilities), processes, and procedures needed to achieve its objectives. We will examine several key diagrams, including the object diagram, state machine diagram, and package diagram, to support our understanding of the system's design and functionality.

I. TECHNICAL BRANCH

A. GENERIC DESIGN

1. Hardware diagram of the system

The hardware diagram simply shows how the system components of our system are deployed; it shows the positioning of each device into its right proportion.

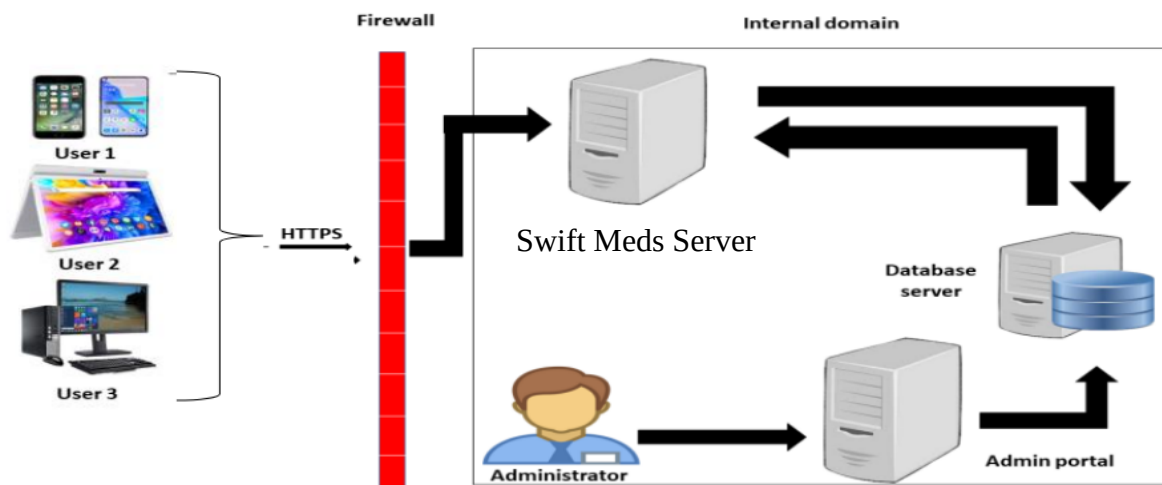


Figure 21: Hardware diagram of the system

2. Description of System Components

Here are the descriptions of the major components of our system:

i. External Users (User1, User 2, User 3):

End users accessing the "Swift Meds" application via a web browser on the public internet. Their requests are sent over HTTPS for security.

ii. Firewall:

The first line of defence, it controls incoming and outgoing network traffic based on predetermined security rules. It filters all the traffic from the internet, only allowing legitimate web traffic to pass through to the internal components, blocking malicious attempts.



iii. Application Server (Swift Meds Server):

The core business logic hub, these servers host the "Swift Meds" web application code. They process user requests, execute operations, and communicate with the database. They receive requests from the load balancer, authenticate users, process orders, and generate dynamic web pages. They then send requests to the database to fetch or update information.

iv. Database Server:

The persistent storage layer. It stores all critical data such as user profiles, product information, inventory, and order history. The Application Servers query this database using secure connections.

v. Administrator Portal:

A specialized interface for system administrators. It may be part of the main application with heightened permissions or a separate internal service. Administrators connect from a secure internal network to manage users, manage pharmacies, view system analytics, and perform maintenance tasks.

B. CAPTURE OF TECHNICAL NEEDS

1. Physical Architecture

The design of the DBMS depends on its architecture. An n-tier architecture partitions on the whole system into related but separated n modules, which can be independently modified, altered, changed or replace. A large amount of data on web servers, personal computers (pc) and others are link with networks with the help of basic client or server architecture.

Within the scope of our project, we made use of the 3-tier architecture. This architecture separate it tiers from each other based upon the user and the manipulated data in the database. Each layer has a well-defined communication interface, and the evolution of the layer is independent of the other. The 3-tier of our system is made up of:

- The client tier, which is also known as our presentation interphase.
- The application tier, which represents our web server.
- The data tier, which represents our DBMS server.

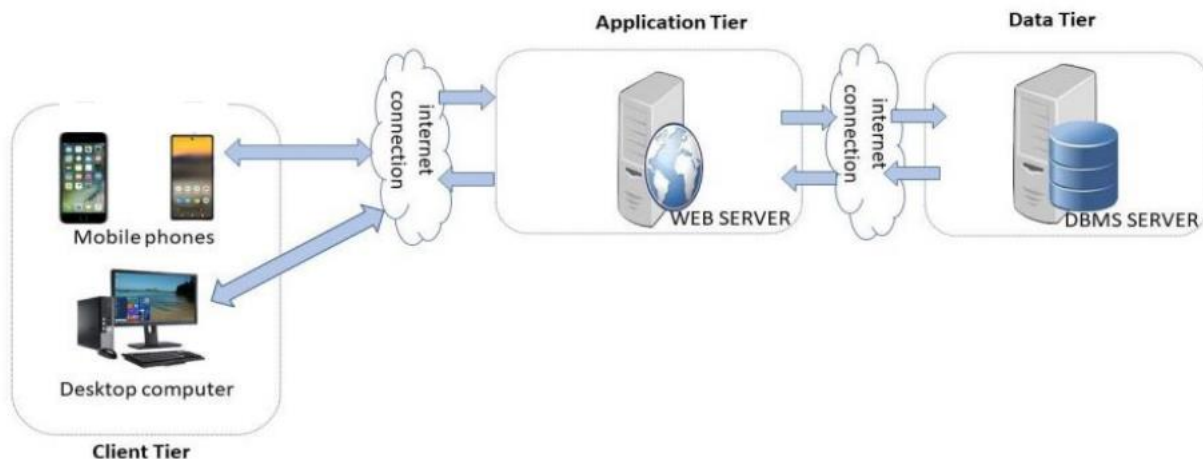


Figure 22: n-tier Architecture (source: <https://www.pinterest.es/pin/752241943987565648/>)

2. Logical Architecture

Model View controller or MVC as it is popularly called, is a software design pattern for developing application. A model view controller pattern is made up of the following three parts.

- **Model:** The lowest level of the pattern which is responsible for maintaining data.
- **View:** This is responsible for displaying all or a portion of data to the user.
- **Controller:** It handles software codes that control the interactions between the model and the view.

MVC is popular as it isolates the application logic from the user interface and supports separation of concerns. Here the controller receives all requests for the application then works with the model to prepare data needed by the view. The view then uses the data prepared by the controller to produce a final response. The MVC can be represented as follows:

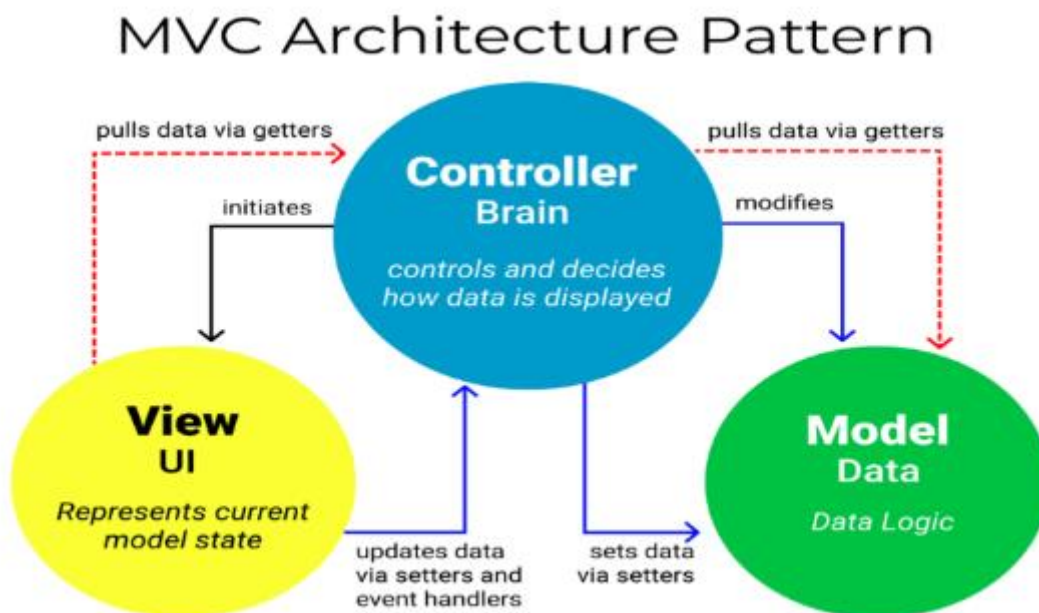


Figure 23: The MVC Architecture (source: <https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/>)

C. RELATED UML DIAGRAMS

1. CLASS DIAGRAM

i. Definition

A class diagram is a static diagram. It represents the static view of an application. class diagram is not only used for visualizing, describing and documenting different aspect of the system but also for constructing executable code of the software application. Class diagram describes the attribute and operation of a class and constraints imposed on the system. Its purpose is to model the static view of an application.

ii. Formalism

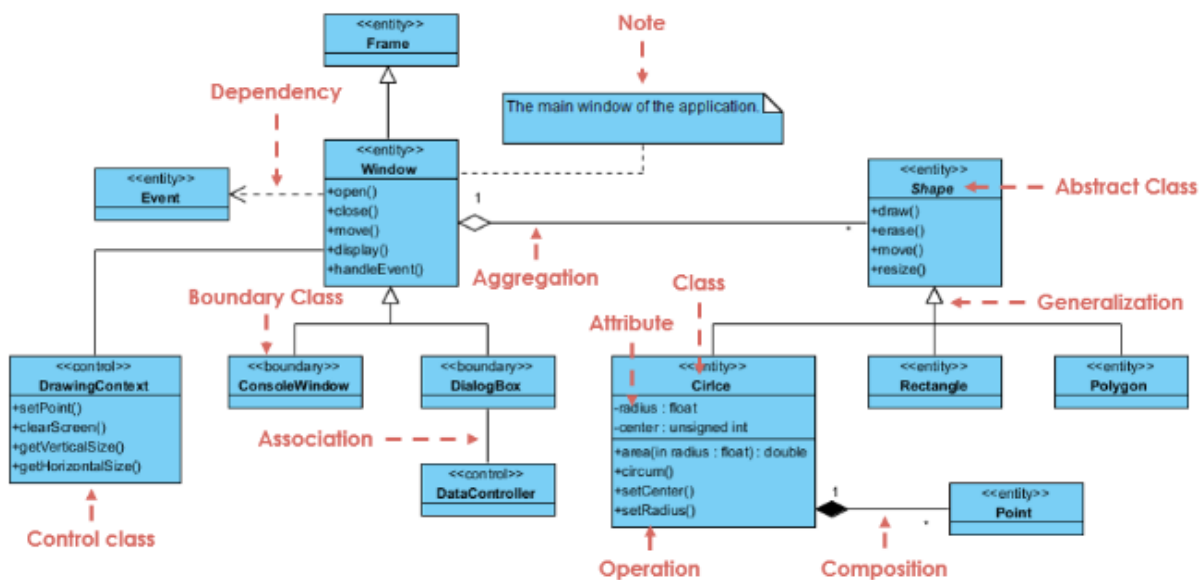
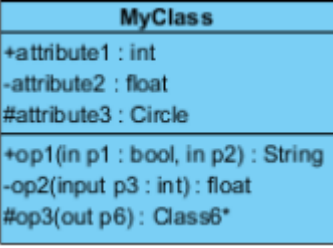
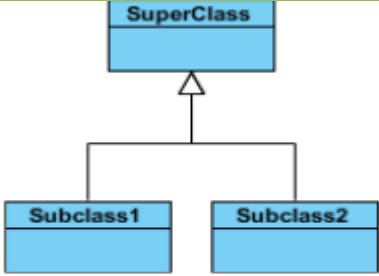
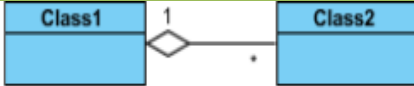
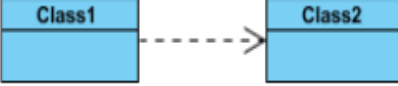


Figure 24: Class diagram formalism

iii. Component of a Class diagram

Table 18: Component of a Class diagram

Element	Notation	Description
Class	 <pre> classDiagram class MyClass { +attribute1 : int -attribute2 : float #attribute3 : Circle +op1(in p1 : bool, in p2) : String -op2(input p3 : int) : float #op3(out p6) : Class6* } </pre>	A class is an element that defines the attributes and behaviors that an object can generate
Generalization	 <pre> classDiagram SuperClass < -- Subclass1 SuperClass < -- Subclass2 </pre>	It is a relationship between a whole thing (called superclass) and a more specific thing (called subclass)
Association	 <pre> classDiagram Class1 -- Class2 </pre>	It is a general type of relationship between elements, it may include cardinality, roles etc.
Aggregation	 <pre> classDiagram Class1 o-- "1" Class2 </pre>	If the parent of the aggregate is deleted, the children are not deleted.
Composition	 <pre> classDiagram Class1 *-- "1" Class2 </pre>	If a parent of a composite is deleted all its parts are deleted with it.
Dependency	 <pre> classDiagram Class1 ..> Class2 </pre>	It exists between two classes, if one changes it may cause the change in the other, but the other way around



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

2. OBJECT DIAGRAM

i. Definition

Object is an instance of a class in a particular moment in runtime that can have its own state and data values. Likewise, a static UML object diagram is an instance of a class diagram, it shows a snapshot of the detailed state of a system at a point in time, thus an object diagram encompasses objects and their relationships which may be considered a special case of a class diagram or a communication diagram.

ii. Formalism

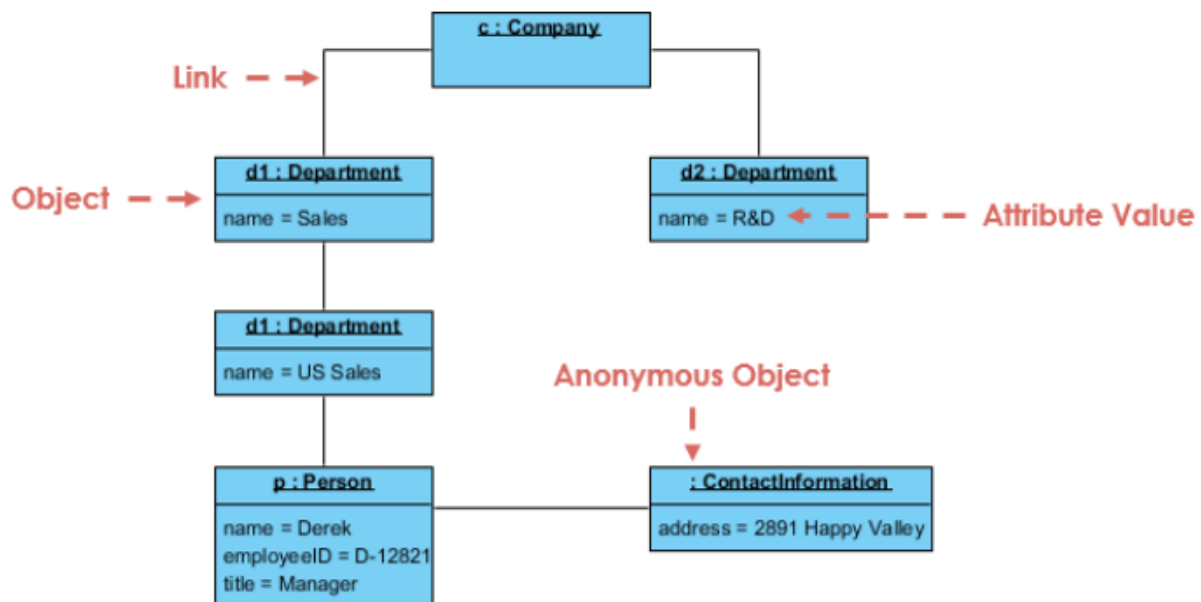
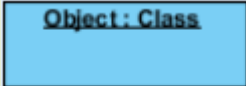
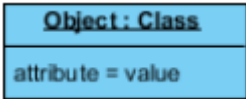
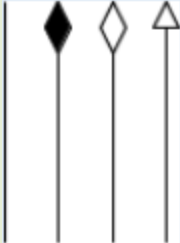


Figure 25: Object diagram formalism

iii. Component of an Object diagram

Table 19: Component of an Object diagram

Element	Notation	Description
Object Names		Every object is actually symbolized like a rectangle, that offers the name from the object and its class underlined as well as divided with a colon.
Object Attributes		Similar to classes, you are able to list object attributes inside a separate compartment. However, unlike classes, object attributes should have values assigned for them.
Links		They represent the temporary or permanent relationships between these specific object at this moment in time.



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

3. STATE MACHINE DIAGRAM

i. Definition

A state machine diagram describes the behaviour of a single object in response to a series of events in a system. Also known as the state machine diagram, it models the dynamic flow of control from the state of a particular object within a system.

ii. Formalism

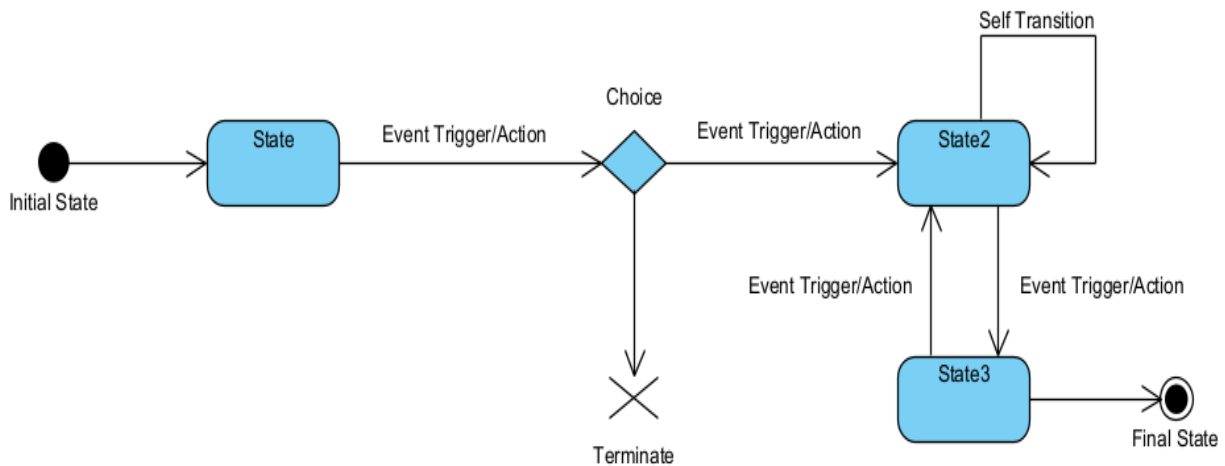
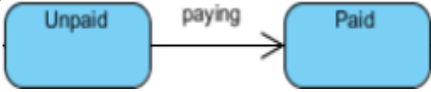


Figure 26: State Machine diagram formalism

iii. Component of a State machine diagram

Table 20: Component of a State machine diagram

Element	Notation	Description
State		Models a situation during which a certain invariant condition holds.
Initial State		It represents a default vertex, that is, a source for a single transaction to the default or composite state.
Final State		A state specifying that the enclosing region is complete.
Transition		A direction relation between a source and a target vertex.
Choice Pseudo State		A diamond symbol that indicates a dynamic condition with branched potential results.
Terminate		Implies that the execution of a state by means of its context is terminated.



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

4. PACKAGE DIAGRAM

i. Definition

This is a structural diagram used to show the organization and arrangement of various model elements in the form of packages. A package diagram is the grouping of related UML elements such as classes, diagrams or even other packages.

ii. Formalism

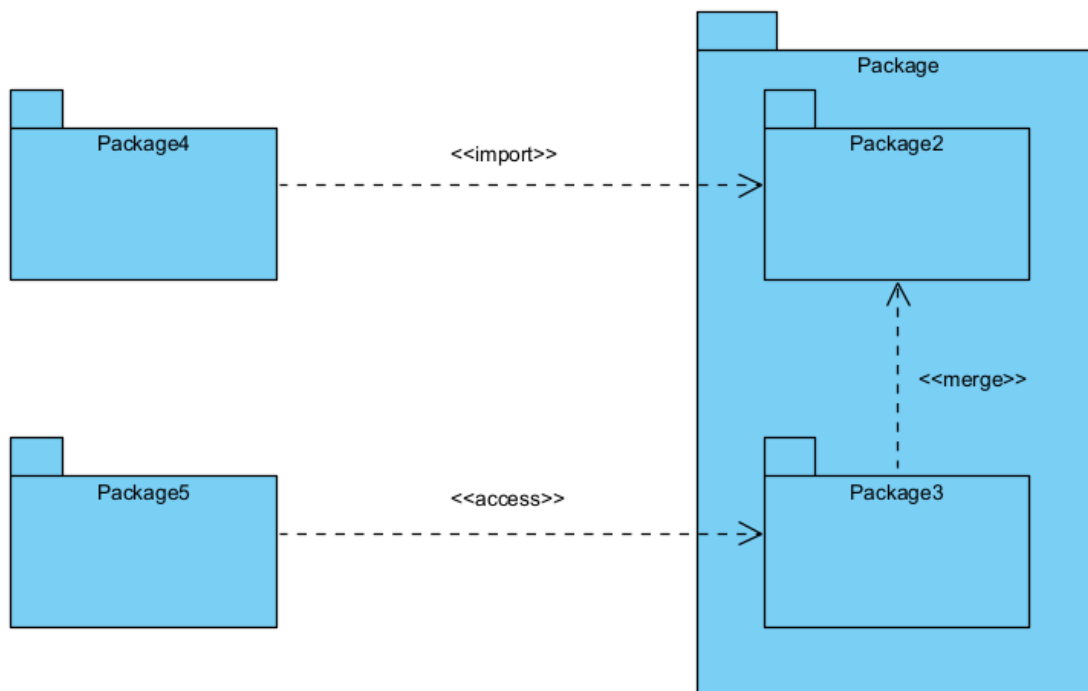
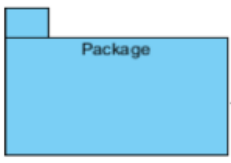
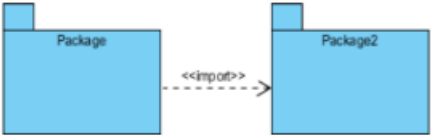
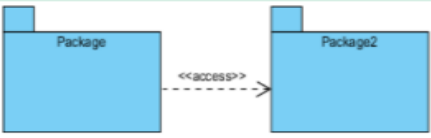


Figure 27: Package diagram formalism

iii. Component of a Package diagram

Table 21: Component of a Package diagram

Element	Notation	Description
Package		A package is a namespace use to group related elements; it is a mechanism used to group elements into a better structure in a system.
Package import		A relationship Indicate that, functionality has been imported from one package to another.
Package access		A relationship Indicates that one package requires assistance from the function of another package.
Package merge		It is a relationship which shows that, the functionality of two packages are combines to a single function.

iv. Swift Meds package diagram

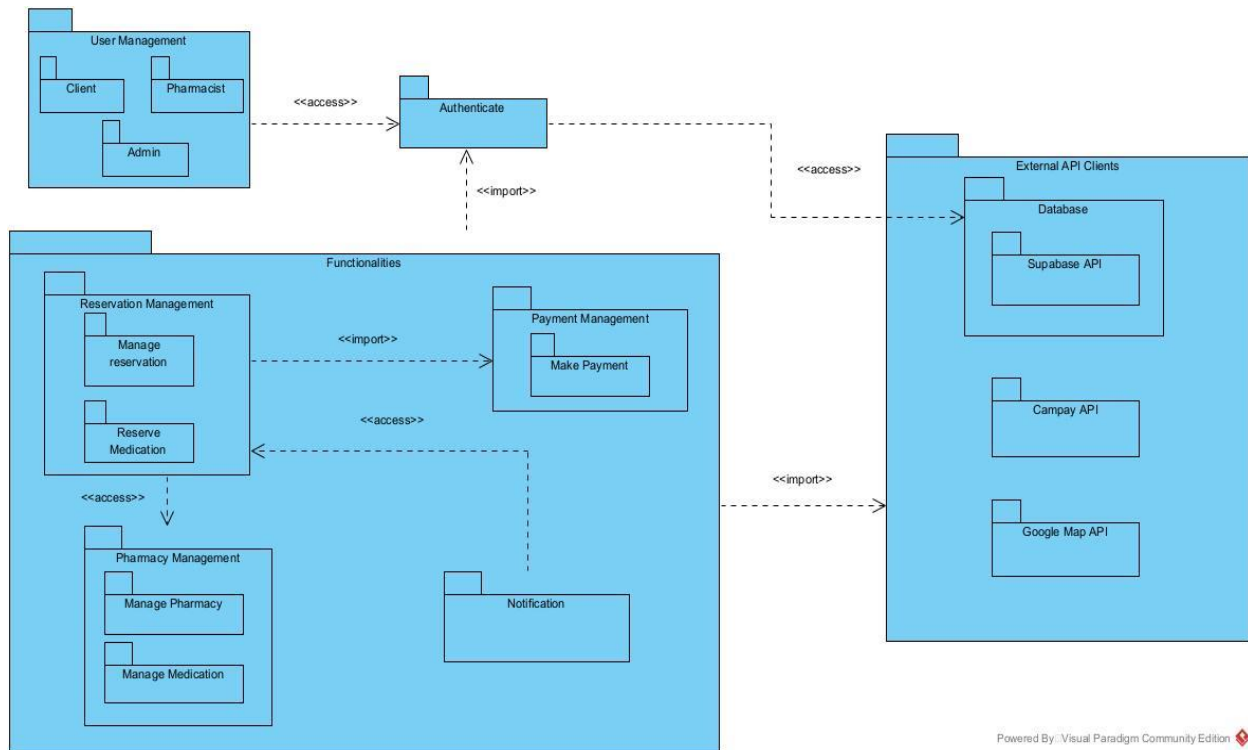


Figure 28: Swift Meds package diagram



CONCLUSION

In the conception phase, our objective was to plan the various aspects of our system, demonstrating how it will be structured and integrated within existing technical architectures. We began by addressing the technical constraints for our system, followed by identifying the components, their groupings, and the appropriate deployment targets for each. We concluded this phase by examining the interactions between the different elements and actors within our system. The next phase of our report is the realization phase, where we will focus on the implementation aspects of our system.



FILE 5: REALIZATION PHASE



PREAMBLE

In this phase we will to straight forward in the implementation of our solution, we will base ourselves on the analysis and conception phases and also present the component and deployment diagrams.

CONTENT

INTRODUCTION

- I. USED TECHNOLOGY
- II. DEPLOYMENT DIAGRAM
- III. COMPONENT DIAGRAM

CONCLUSION

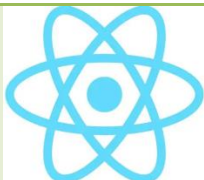


INTRODUCTION

The realization phase marks the transition from design to implementation for our application, focusing on the technologies used and the relevant UML diagrams, specifically the deployment and component diagrams. We used different technologies in the confection of our application (Front-end, Back-end and Database). The deployment diagram will illustrate the physical architecture of the application components across servers and services, while the component diagram will detail the internal structure, highlighting key components and their interactions. This approach aims to direct the realization of our system to the project goal.

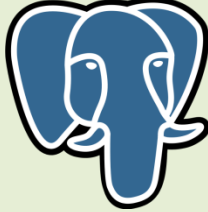
I. USED TECHNOLOGY

Table 22: Used Technology

Technology	Logo	Description
Visual Studio Code		Visual Studio Code is a lightweight, open-source code editor developed by Microsoft, supporting multiple programming languages. Using for writing of application code
Microsoft Word		Microsoft Word is a widely- used word processing application that allows users to create, edit, and format text documents. Using in application report creation.
Microsoft PowerPoint		Microsoft PowerPoint is a presentation software that allows users to create dynamic slideshows comprising text, images, videos, and animations. Able us to create slideshows presentations for application.
Git		Git is a distributed version control system designed to manage and track changes in source code during software development. Using to store ours code through the development process.
React		A popular JavaScript library for building UI. It provides a component- based approach and allows for efficient rendering and state management. Used

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

		for building the Frontend
Node JS		Node.js is an open-source, cross-platform JavaScript runtime environment that allows developers to execute JavaScript code outside of a web browser. Used for building the Backend
PostgreSQL		PostgreSQL, often referred to as Postgres, is a powerful, open-source, object-relational database management system. Used for the database of the system
Supabase		Supabase is the Postgres development platform that powers a new generation of developer tools. Used for the hosting of the database
Firefox		Firefox is a free and open-source web browser developed by the Mozilla Foundation and its subsidiary, the Mozilla Corporation. Used to test the UI (web pages) of Swift Meds
Postman		Postman is a popular collaboration platform for API development that allows users to design, test, and document APIs. Used to test our backend API
Visual Paradigm		Visual Paradigm is a comprehensive modeling tool that supports software design and system modeling. Used to design our UML diagrams.

Icogram		Icogram is a social media platform designed for sharing and discovering visual content, particularly focused on images and photography. Used to draw our company geographical location.
Gantt Project		Gantt Project is a free, open-source project management tool that allows users to create Gantt charts for planning and scheduling projects
Vercel		Vercel is a cloud platform designed for deploying and scaling web applications, particularly those built with popular JavaScript frameworks like React or Next.js. Used to deploy Swift Meds

II. DEPLOYMENT DIAGRAM

A. Definition

Deployment diagram is a structural diagram used to visualize the topology of the physical components of a system, where the software is deployed. They consist of nodes and their relationship. It is related to the component diagram because the components are deployed using the deployment diagram. A deployment diagram consists of nodes. Nodes are nothing but physical hardware used to deploy the application.

B. Formalism

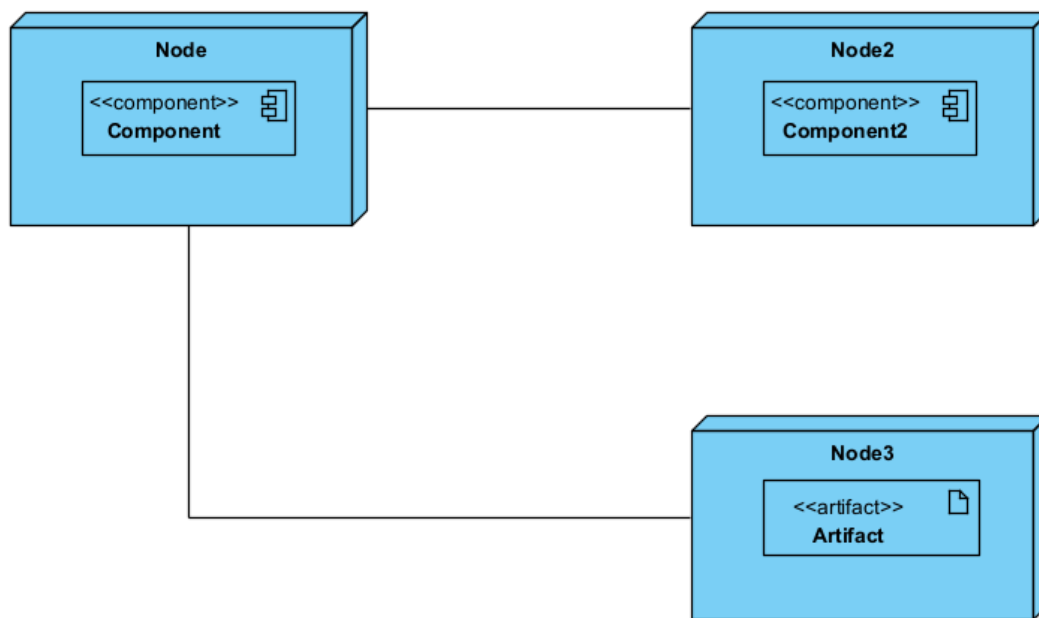
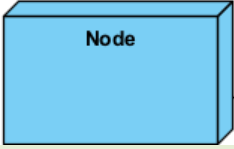
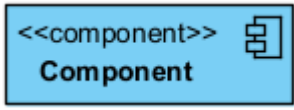


Figure 29: Deployment diagram formalism

C. Component of a Deployment diagram

Table 23: Component of a Deployment diagram

Element	Notation	Description
Node		A hardware or software used to deploy the application.
Artifact		An artifact is a major product, which is produced or used during the development of software. E.g. diagrams, data models, setup scripts.
Component		It represents a modular part of a system that encapsulates its content and whose manifestation is replaceable within its environment.
Association		An association helps to connect two nodes together which permit them to communicate together.

D. Swift Meds deployment diagram

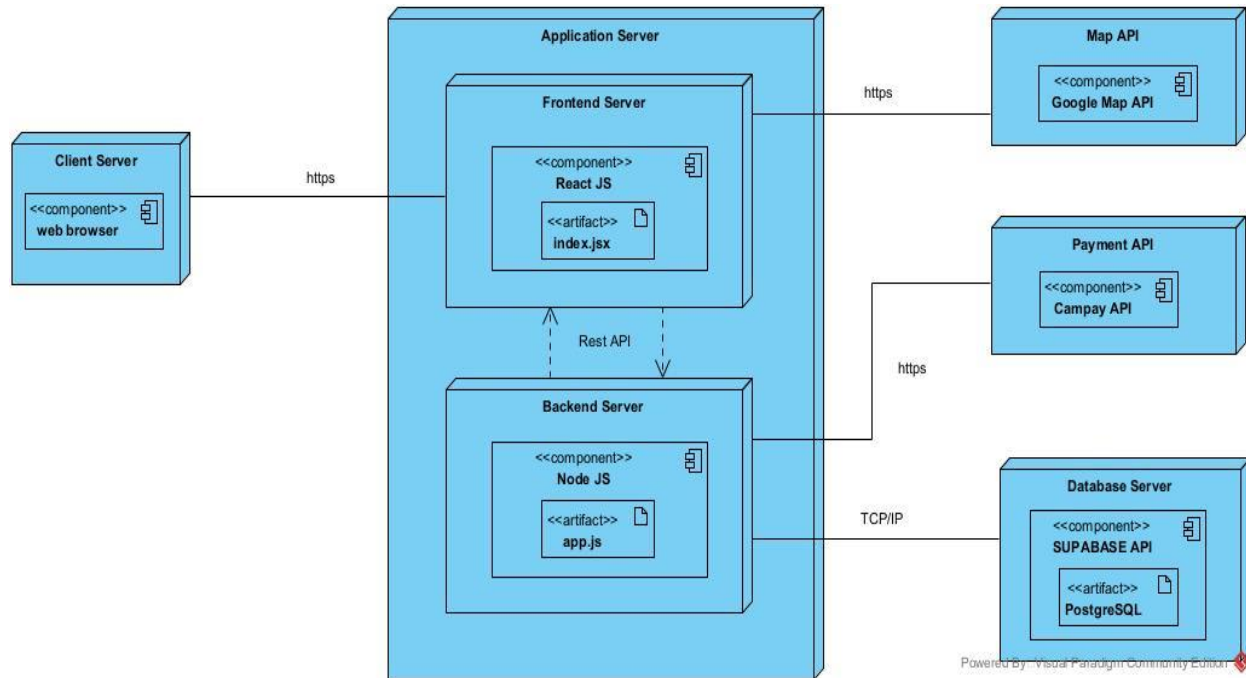


Figure 30: Swift Meds deployment diagram

III. COMPONENT DIAGRAM

A. Definition

Component diagrams are used to model the physical aspect of a system. Now the question is what is this physical aspect? They are elements such as Executables, libraries, files, document etc. which resides in a node. The component diagram does not describe the functionality of the system, but it describes the components used to make those functionalities.

B. Formalism

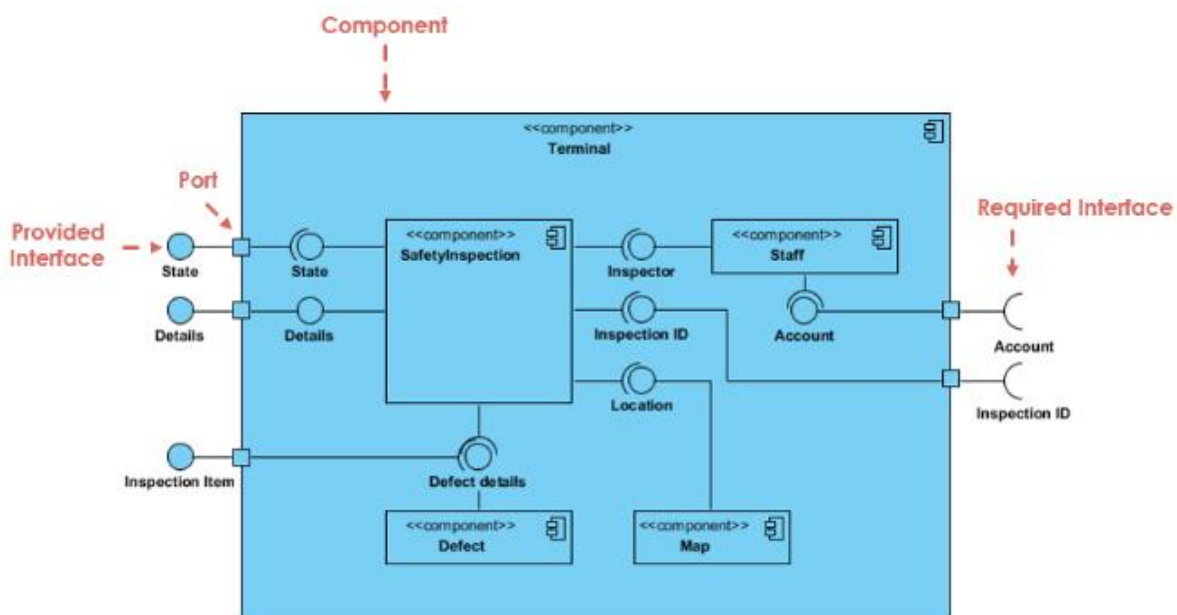
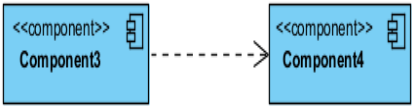
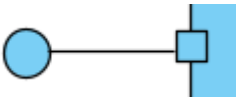
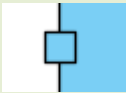


Figure 31: Component diagram formalism

C. Component of Component diagram

Table 24: Component of a Component diagram

Element	Notation	Description
Component		A component is an abstract logical unit block of a system.
Dependency		Dependency is a directed relationship which is used to show that some components are dependent on others for their correct functioning.
Required Interface		It is a straight line from the component box with an attached half circle representing interfaces where a component requires information in order to perform its own functions
Provided Interface		It is a straight line from the component box with an attached circle representing interfaces where a component produces information used by required interfaces
Port		A port is used when the components delegate the interfaces to an internal class.

D. Swift Meds component diagram

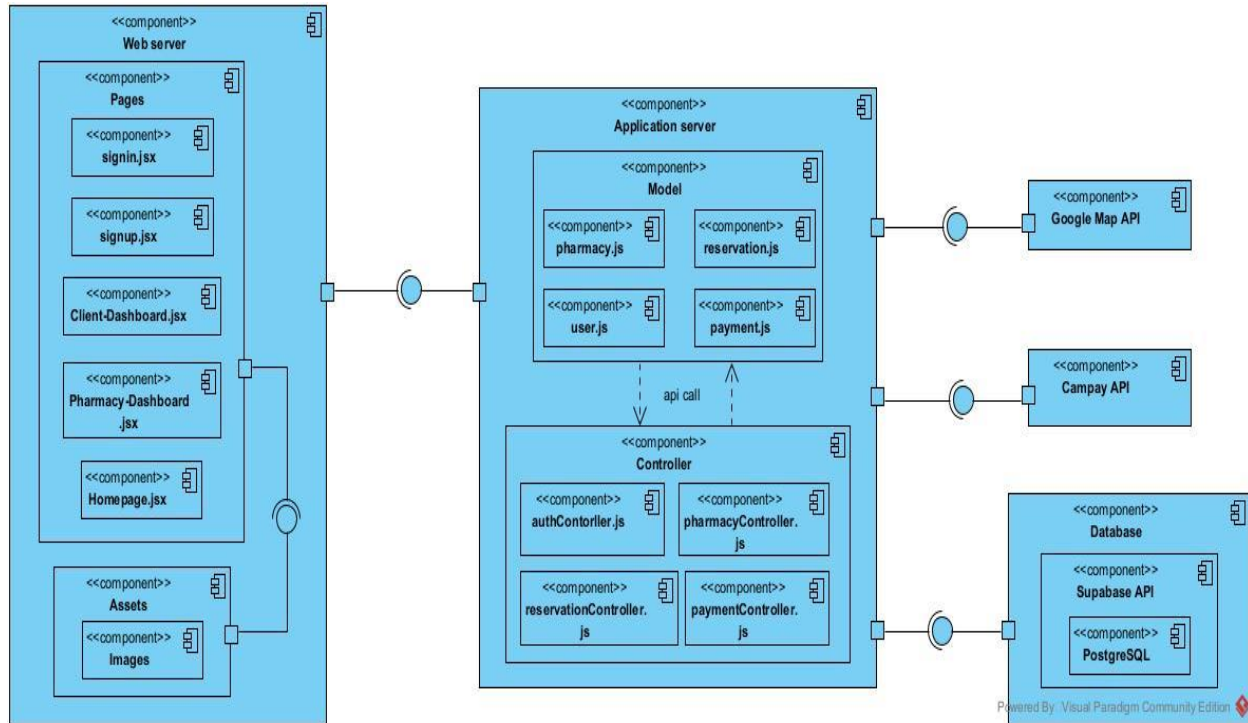


Figure 32: Swift Meds component diagram



CONCLUSION

In conclusion, the realization phase is a crucial step for the development of our application. To achieve this, we utilized our analysis and conception phase. We also created deployment and component diagrams which illustrate the modules, files, and assets, as well as how different elements interact with each other within our system. Our next step is to move onto the functionalities testing phase, where we will assess the efficacy of the various modules present in our app for different users.



FILE 6: TEST OF FUNCTIONALITIES



PREAMBLE

In this phase, we will present the different test of functionalities of our system.

CONTENT

INTRODUCTION

- I. TEST OF FUNCTIONALITIES
- II. TEST SHOWCASE

CONCLUSION



INTRODUCTION

The Test of Functionalities represents a critical phase in the development lifecycle of the Swift Meds system, dedicated to systematically verifying that all software components operate according to specified requirements and user expectations. This comprehensive testing strategy employs a multi-layered approach, utilizing specialized tools including Jest for unit testing business logic verification.



I. TEST OF FUNCTIONALITIES

A. What is Unit Testing?

Unit testing involves testing the smallest parts of an application, such as functions or methods, independently from the rest of the application. The primary goal is to validate that each unit of the software performs as expected.

B. Unit Testing Importance

Unit testing ensures that each component of the application works independently and correctly. It helps catch bugs early in the development process and guarantees that future changes don't break existing functionality.

C. Testing Strategy

We implemented unit testing to isolate and test individual parts of the app, specifically the API endpoints and backend services. Unit testing provides fast feedback on code quality and behaviour.

D. Tool Used

- **Jest:** Testing framework used for running unit tests.

E. Key Functionalities Tested

User Registration

- Tests ensure that users can successfully register with valid details.
- Handles error cases where invalid inputs or missing fields cause registration failure.

User Authentication

- Tests confirm that users can log in with correct credentials.
- Includes scenarios where login fails due to invalid credentials (e.g., wrong password).

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025

II. TEST SHOWCASE

A. Registration Showcase

- User Registration: Demonstrates successful user registration with valid data.

```
describe('AuthService', () => {
  describe('signUp', () => {
    it('should create a new user successfully', async () => {
      const mockUserData = {
        email: 'percy@example.com',
        password: 'password123',
        fullName: 'Test User',
        role: 'client'
      };

      const mockChain = {
        insert: jest.fn().mockReturnThis(),
        select: jest.fn().mockReturnThis(),
        single: jest.fn().mockResolvedValue({
          data: { user_id: 'test-id', ...mockUserData },
          error: null
        })
      };

      supabase.from.mockReturnValue(mockChain);

      const result = await AuthService.signUp(mockUserData);

      expect(result.success).toBe(true);
      expect(result.user).toBeDefined();
      expect(result.user.email).toBe('percy@example.com');
    });

    it('should handle signup errors', async () => {
      const mockError = { message: 'Email already exists' };

      supabase.from.mockReturnValue({
        insert: jest.fn().mockReturnThis(),
        select: jest.fn().mockReturnThis(),
        single: jest.fn().mockResolvedValue({ data: null, error: mockError }),
      });

      await expect(
        AuthService.signUp({
          email: 'percy@example.com',
          password: 'password123',
          fullName: 'Test User',
          role: 'client'
        })
      ).rejects.toThrow('Email already exists');
    });
  });
});
```

Figure 33: User registration code test snippet

B. Authentication Showcase

- User Authentication: Shows successful login and token generation when valid credentials are provided.

```
describe('AuthService', () => {
  // ...

  describe('signIn', () => {
    it('should sign in a user successfully', async () => {
      const mockUser = {
        email: 'percy@example.com',
        password_hash: 'password123',
      };

      supabase.from.mockReturnValue({
        select: jest.fn().mockReturnThis(),
        eq: jest.fn().mockReturnThis(),
        single: jest.fn().mockResolvedValue({ data: mockUser, error: null }),
      });

      const result = await AuthService.signIn('percy@example.com', 'password123');

      expect(result.success).toBe(true);
      expect(result.user).toBeDefined();
      expect(result.token).toBeDefined();
    });

    it('should handle invalid credentials', async () => {
      supabase.from.mockReturnValue({
        select: jest.fn().mockReturnThis(),
        eq: jest.fn().mockReturnThis(),
        single: jest.fn().mockResolvedValue({ data: null, error: new Error('User not found') }),
      });

      await expect(
        AuthService.signIn('percy12@example.com', 'password123')
      ).rejects.toThrow('Invalid credentials');
    });

    /* it('should handle incorrect password', async () => { ...
  });
});
```

Figure 34: Authentication test code snippet



```
PASS tests/services/auth_service.test.js (5.995 s)
AuthService
  signUp
    ✓ should create a new user successfully (2257 ms)
    ✓ should handle signup errors (966 ms)
  signIn
    ✓ should sign in a user successfully (1195 ms)
    ✓ should handle invalid credentials (337 ms)
```

Figure 35: Registration and Authentication unit test



CONCLUSION

In conclusion, the unit testing phase, rigorously conducted with the Jest framework, has proven to be the cornerstone of validating the Swift Meds system's functional integrity. By isolating and examining the smallest testable units of code from the “medication search algorithms and distance calculation utilities to the reservation logic and inventory management rules,” we have systematically verified that each individual component performs its designated function with precision and reliability.



FILE 7: INSTALLATION AND USER GUIDE



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



PREAMBLE

The purpose of the user guide is to provide users of our platform with step-by-step instructions on how to install and use the system.

CONTENT

INTRODUCTION

I. INSTALLATION OF THE APPLICATION

II. SHOWCASE

CONCLUSION



INTRODUCTION

This Installation and User Guide provides comprehensive instructions for deploying and utilizing the Swift Meds platform, a real-time pharmacy management system designed to streamline medication access in Yaoundé, Cameroon. This guide provides step-by-step instructions to install and configure the Swift Meds system. It covers environment setup, dependency installation, and all procedures to get your pharmacy system platform running successfully.



I. INSTALLATION OF THE APPLICATION

A. System Requirements

1. For Developers:

- **Operating System:** Windows, Mac OS, or Linux
- **Node.js:** Version 14.x or later
- **npm:** Latest version (for package management)
- **Git** (for cloning the repository)
- **Visual Studio Code** or any other preferred text editor/IDE
- **Supabase Project** (for the database)

2. For Users:

- **Device:** Android phone, Desktop or Laptop computer
- **Internet Connection**

B. Installation Guide (For Developers)

1. Backend Installation

Step1: Clone the repository

- git clone https://github.com/kekelo17/Swift_Meds.git
- cd Back-end

Step 2: Install dependencies

- npm install

Step 3: Set up environment variables

- Create a **.env** file in the root of your backend project folder and add the following:



```
PORT=your_port
SUPABASE_URL=your_supabase_url
SUPABASE_ANON_KEY=your_supabase_anon_key
JWT_SECRET=your_jwt_secret_key
FRONTEND_URL=http://localhost:3000
GEOCODE_API_KEY=your_geocode_api_key
```

Figure 36: frontend configuration of the .env file

- Replace with your variables

Step 4: Run the server

- npm run dev

2. Frontend Installation

Step1: Enter the frontend directory

- cd Front-end

Step 2: Install dependencies

- npm install

Step 3: Set up environment variables

- Create a **.env** file in the frontend project and configure the backend VITE_API_URL

```
VITE_API_URL=http://localhost:5000/api
VITE_GOOGLE_MAP_API_KEY=your_google_map_api_key
VITE_GEOAPIFY_API_KEY=your_geoapify_api_key
```

Figure 37: backend configuration of the .env file

- Replace with your variables

Step 4: Start the app, run the following command to start the web server for the app

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



- `npm run dev`

C. Installation Guide (For Users)

- Visit the Swift Meds website: <https://swift-meds.vercel.app>
- Create an account to log in the system



II. SHOWCASE

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



CONCLUSION

In conclusion, this Installation and User Guide has equipped both development teams and end-users with the necessary knowledge to successfully deploy and interact with the Swift Meds system.



GENERAL CONCLUSION

In conclusion, this project stands as a significant milestone in the intersection of technology and healthcare, successfully delivering a sophisticated, user-centric pharmacy platform that fundamentally redefines the user experience. It transcends the basic functionality of medication searches by building a comprehensive, integrated ecosystem—a digital sanctuary for health that seamlessly connects clients with pharmacies through intelligent filtering, transparent community-driven reviews, and secure, streamlined transactions. This initiative represents far more than a technological achievement; it is a heartfelt commitment to community well-being, transforming the often stressful and fragmented task of managing personal and family health into an intuitive, efficient, and empowering act of care. The strategic incorporation of a premium subscription model and diverse payment gateways like MOMO not only enhances user convenience and fosters deep engagement but also establishes a robust, scalable, and sustainable business model for the future. Ultimately, by positioning the application as both a future-ready leader in digital healthcare accessibility and a compassionate, trusted partner in the user's health journey, we have laid the foundation for a platform that not only meets immediate needs but also nurtures long-term wellness and peace of mind for the entire community it serves.



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



ANNEXES

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



BIBLIOGRAPHY

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



REFERENCES



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



WEBOGRAPHY

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



TABLE OF CONTENT

DEDICATION	i
ACKNOWLEDGMENTS	ii
CONTENT	iii
LIST OF FIGURES	iv
LIST OF TABLES	vi
GLOSSARY	vii
ABSTRACT.....	viii
RESUME	ix
GENERAL INTRODUCTION.....	1
PART I: INSERTION PHASE	2
PREAMBLE	3
CONTENT	3
INTRODUCTION	4
I. WELCOME AND INSERTION.....	5
A. Background.....	6
B. Missions, Visions and Activities of Realize	6
1. Missions:	6
2. Visions:	6
3. Activities:	6
4. Localization:	7
III. ORGANIZATION OF THE COMPANY	8
A. Administrative Organization.....	8
1. The General Management.....	8
2. Human Resource Department	8
3. Communication Department	8
4. Department of Financial Affairs	9
5. Technical Department.....	9
6. Software Engineering Department.....	9

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



B.	Functional Organization of REALIZE.....	10
IV.	HARDWARE AND SOFTWARE RESOURCES	11
A.	HARDWARE RESOURCES	11
B.	SOFTWARE RESOURCES	11
V.	BRIEF PRESENTATION OF PROJECT THEME	12
	CONCLUSION.....	13
	PART II: TECHNICAL PHASE	14
	FILE 1: EXISTING SYSTEM.....	15
	PREAMBLE	16
	CONTENT	16
	INTRODUCTION	17
I.	PRESENTATION OF THE THEME	18
II.	STUDY OF THE EXISTING SYSTEM	19
III.	LIMITATIONS OF THE EXISTING SYSTEM.....	20
IV.	PROBLEMATICS	21
V.	PROPOSED SOLUTION	22
i.	Centralized Web Platform:.....	22
ii.	Real-Time Medication Availability:	22
iii.	Facilitation of Fast Dispensing:	22
iv.	Critical Intermediary:.....	22
v.	User-Friendly Intuitive Interface:	22
	CONCLUSION.....	23
	FILE 2: SPECIFICATION BOOK	24
	PREAMBLE	25
	CONTENT	25
	INTRODUCTION	26
I.	CONTEXT AND JUSTIFICATION	27
A.	CONTEXT	27
B.	JUSTIFICATION	28
1.	Operational Necessity:	28
2.	Public Health Imperative:	28
3.	Economic Imperative:.....	28

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



4. Technological Imperative:	28
II. OBJECTIVES OF THE PROJECT	29
A. GENERAL OBJECTIVE.....	29
B. SPECIFIC OBJECTIVES	29
III. EXPRESSION OF NEEDS	30
A. FUNCTIONAL NEEDS	30
1. User Management	30
2. Search and Discovery.....	30
3. Reservation and Payment.....	30
4. Pharmacy Management.....	31
5. Analytics and Reporting.....	31
6. Notification System.....	31
B. NON-FUNCTIONAL NEEDS	31
1. Usability.....	31
2. Performance	32
3. Reliability and Availability	32
4. Security	32
5. Scalability	32
6. Maintainability	32
IV. PROJECT PLANNING	33
A. Chronogram of Activities.....	33
B. Gantt Project Planning	34
V. ESTIMATED COST OF THE PROJECT	35
A. Software Resources.....	35
B. Hardware Resources	36
C. Human Resources	36
D. Global Estimation	37
VI. PROJECT CONSTRAINTS	38
A. Technical Constraint	38
B. Time Constraint	38
C. Cost Constraint.....	38
VII. LIST OF PARTICIPANTS AND DELIVERABLES.....	39
A. LIST OF PARTICIPANTS.....	39

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



B. DELIVERABLES	39
CONCLUSION.....	40
FILE 3: ANALYSIS BOOK.....	41
PREAMBLE	42
CONTENT	42
INTRODUCTION	43
I. COMPARATIVE STUDY OF UML AND MERISE	44
A. MERISE.....	44
B. UML.....	44
1. STATIC OR STRUCTURAL DIAGRAMS.....	44
2. BEHAVIORAL OR DYNAMIC DIAGRAMS.....	45
II. COMPARATIVE STUDY OF UNIFIED PROCESS	47
A. Unified Process	47
B. Two Track Unified Process (2TUP)	47
1. The Left Branch (Functional branch).....	48
2. The Right Branch (Technical branch).....	48
3. The Middle Branch (Realization or Implementation branch)	48
III. CHOICE OF THE ANALYSIS METHOD	50
IV. MODELLING THE PROPOSED SOLUTION.....	51
A. Capture of the Functional Needs.....	51
1. USE CASE DIAGRAM.....	51
2. Textual Description.....	55
3. ACTIVITY DIAGRAM.....	57
4. SEQUENCE DIAGRAM.....	63
5. COMMUNICATION DIAGRAM.....	68
CONCLUSION.....	72
FILE 4: CONCEPTION PHASE	73
PREAMBLE	74
CONTENT	74
INTRODUCTION	75
I. TECHNICAL BRANCH	76
A. GENERIC DESIGN.....	76

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



1. Hardware diagram of the system.....	76
2. Description of System Components.....	76
B. CAPTURE OF TECHNICAL NEEDS.....	78
1. Physical Architecture	78
2. Logical Architecture	79
C. RELATED UML DIAGRAMS	80
1. CLASS DIAGRAM	80
2. OBJECT DIAGRAM.....	83
3. STATE MACHINE DIAGRAM	86
4. PACKAGE DIAGRAM.....	89
CONCLUSION.....	92
FILE 5: REALIZATION PHASE.....	93
PREAMBLE	94
CONTENT	94
INTRODUCTION	95
I. USED TECHNOLOGY	96
II. DEPLOYMENT DIAGRAM	99
A. Definition	99
B. Formalism	99
C. Component of a Deployment diagram	100
D. Swift Meds deployment diagram	101
III. COMPONENT DIAGRAM.....	102
A. Definition	102
B. Formalism	102
C. Component of Component diagram.....	103
D. Swift Meds component diagram	104
CONCLUSION.....	105
FILE 6: TEST OF FUNCTIONALITIES	106
PREAMBLE	107
CONTENT	107
INTRODUCTION	108
I. TEST OF FUNCTIONALITIES.....	109

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



A. What is Unit Testing?	109
B. Unit Testing Importance	109
C. Testing Strategy	109
D. Tool Used.....	109
E. Key Functionalities Tested.....	109
User Registration	109
User Authentication	109
II. TEST SHOWCASE	110
A. Registration Showcase	110
B. Authentication Showcase	111
CONCLUSION.....	113
FILE 7: INSTALLATION AND USER GUIDE.....	114
PREAMBLE	115
CONTENT	115
INTRODUCTION	116
I. INSTALLATION OF THE APPLICATION.....	117
A. System Requirements.....	117
1. For Developers:.....	117
2. For Users:.....	117
B. Installation Guide (For Developers)	117
1. Backend Installation.....	117
2. Frontend Installation	118
C. Installation Guide (For Users).....	119
II. SHOWCASE.....	120
CONCLUSION.....	121
GENERAL CONCLUSION	122
ANNEXES	x
BIBLIOGRAPHY	xi
REFERENCES	xii
WEBOGRAPHY	xiii
TABLE OF CONTENT	xiv

KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025



DEVELOPMENT OF A PHARMACY SYSTEM FOR REAL-TIME MEDICATION AVAILABILITY AND FAST DISPENSING



KEUMOE KENNY LOANN

African Institute of Computer Sciences (AICS) Cameroon Paul BIYA Technological Centre of
Excellence, Second Year of Software Engineering, Academic Year 2024 – 2025